Databases project 2025-2026

Data Analysis of the Paris 2024 Olympic Games

Member list (name and email): Alex Vilamitjana ( ), Oriol Pros ( ), Sara Gibert ( )

Dataset: Olympic Games 2024

Delivery date: 17/05/2026

Index

1 INTRODUCTION (1-2 PAGES) 3
1.1 PROJECT TEAM 4
1.2 DATASET DESCRIPTION 4
1.3 VALIDATION QUESTIONS 5
2 ENTITY-RELATIONSHIP MODEL 7
2.1 DIAGRAM 7
2.2 JUSTIFICATION (4-6 PAGES) 7
3 RELATIONAL MODEL 14
3.1 DIAGRAM 14
3.2 JUSTIFICATION (1-3 PAGES) 14
4 DATATYPE SELECTION (1-2 PAGES) 16
5 PHYSICAL MODEL CODIFICATION (1-2 PAGES) 19
6 DATA IMPORT (2-3 PAGES PER FILE) 20
6.1 NOC.CSV 20
6.2 VENUES.CSV 20
6.3 EVENTS.CSV 21
6.4 SCHEDULES.CSV 22
6.5 ATHLETES.CSV 23
6.6 COACHES.CSV 25
6.7 TECHNICAL_OFFICIALS.CSV 26
6.8 MEDALS.CSV 27
6.9 MEDALLISTS.CSV 28
6.10 TEAMS.CSV 29
6.11 TRAINING_SESSIONS.CSV 30
7 QUERY IMPLEMENTATION 32
7.1 WHICH ATHLETES HAVE WON MORE MEDALS THAN THE AVERAGE NUMBER OF MEDALS WON PER ATHLETE ACROSS ALL DISCIPLINES? 32
7.2 WHICH COUNTRIES HAVE MANAGED TO WIN MEDALS IN ALL THREE CATEGORIES FOR INDIVIDUAL ATHLETES: GOLD, SILVER, AND BRONZE? 33

7.3 Which coaches are currently managing more athletes than the average number of athletes per coach? 35
7.4 Which 5 disciplines have a higher number of female participants than male participants? 37
7.5 Which athletes have managed to win medals and participate in more than 10 unique events? 38
7.6 Which venues have hosted events belonging to more than one different sport? 40
7.7 Which athletes representing Spain have also studied in University at Spain? 42
7.8 Which coaches are training athletes across more than one discipline at the same time? 44
7.9 Which countries have sent more athletes to compete than the average number of athletes per country? 45
7.10 Which sports have had their events spread across the higher number of different venues? 47
7.11 Which athletes share the same personal hero (father) and have also won at least one medal? 48
7.12 Which disciplines include both athletes born in the same country they represent and athletes born in a different country from the one they represent? 50
7.13 Which athletes were born in the same country they're representing and have won at least one gold medal? ... 52
7.14 Which disciplines have an average athlete age that is higher than the global average age across all disciplines? 53
7.15 Which organizations have technical officials involucrate in more than ten different disciplines? 55
7.16 Which athletes have won at least one medal and also have a sister who also does sports? 56
7.17 Which countries have athletes competing in every discipline currently scheduled in the Olympic Games? 58
7.18 Which coaches have never had any of their athletes win a medal throughout the competition? 59
7.19 Which nationalities appear most frequently among the athletes who have won gold medals? 60
7.20 Which disciplines have the lowest total number of athletes registered to compete? 62
7.21 Which 3 days in the schedule have the highest amount of scheduled events starting at the same time? 63
7.22 Which athletes are currently living in a country different from the one they're officially representing? 64
7.23 Which sports have had all their events concentrated in the fewest number of venues? 66
7.24 Which venues have hosted the highest number of different events within a single competition day? 67
7.25 Which countries have the largest number of athletes competing but haven't won any medals so far? 68
7.26 Which countries have won more team gold medals than the combined total of their team silver and bronze medals? 70
7.27 Which countries have both individual medalists and team medalists? 71
7.28 Which sport is the most trained among athletes that have won a medal? 74
7.29 Which venues have had at least one scheduled event on both the first and the last day of the Olympics? 76
7.30 Which are the top 3 countries with the highest number of athlete medallists whose training sessions have an intensity level above 7? 78

8 TRIGGER IMPLEMENTATION 79

8.1 Trigger 1 79
8.2 Trigger 2 82
8.3 Trigger 3 85

9 CONCLUSIONS 89

9.1 Utilization of resources (1 page) 89
9.2 AI usage (if needed, 1-2 pages) 89
9.3 Lessons learned and conclusions (1 page) 92

1 Introduction (1-2 pages)

Athletes, nations, events, and outcomes are just a few of the many dimensions of data generated by the Olympic Games. However, despite its richness and potential value, extracting meaningful insights from this data is not a straightforward task. In its raw form, the dataset is typically distributed across multiple files, where relationships between different elements are not explicitly defined. This fragmentation makes it difficult to combine information efficiently and limits the type of analysis that can be performed.

We are addressing this challenge by transforming all datasets compounding the Olympic Games 2024, into a structured database. The goal is not only to store the data, but to make it accessible, consistent, and suitable for answering complex analytical questions.

To achieve this, the project focuses on the design and implementation of a database system following a structured process. Specifically, in this first phase, a conceptual model is developed to represent the main entities and relationships within the dataset. It is expected to have a big number of entities and relations since we will we working with a big dataset. Also, this model is then translated into a relational model that complies with normalization principles, up to the 3rd normal form, which ensures data integrity and minimizes redundancy.

Another objective is to explore the analytical potential of the dataset. By querying the database, we will get to know the date we're working with, which will make it possible to answer relevant questions, such as identifying the most successful athletes or countries, analysing medal distributions over time, or comparing performance across different sports. These insights will demonstrate how structured data can be transformed into valuable knowledge. For this phase, we will only make these questions, so that it is demonstrated we have understood the Olympic Games dataset, which will be answered throughout the development of further phases of the project.

On the other hand, our motivation to do this project is the chance to engage with diverse and historically relevant real-world data. Information about individuals, nations, sports, events, and medals from various Games is included in the Olympic Games dataset. Managing many-to-many relationships, maintaining data consistency, and organizing the data to facilitate effective querying are some of the difficulties brought forth by this diversity. Also, we believe this dataset is especially well-suited for using fundamental database concepts since it addresses these issues.

Also, the actual application of the theoretical knowledge learned in this subject is another significant source of motivation. In database systems, ideas like entity-relationship modelling, normalization, and relational schema design are essential. Therefore, through this project, these ideas are put into practice and their efficacy in handling a complicated dataset is assessed, so that we can demonstrate our knowledge learned in class so far.

Taken together, the objectives of this phase and our motivation to accomplish them will allow us to build a robust and efficient database system that not only organizes the Olympic Games data but also enables meaningful analysis, demonstrating the practical importance of database systems in real-world scenarios.

1.1 Project team

A team of three people worked on this project, each of them contributed in a different way to the database design and implementation process. To maintain close communication on all important decisions and to ensure an effective workflow, the job has been divided.

- Sara Gibert was principally in charge of the database's conceptual model. This involved defining constraints, identifying important entities and relationships, and developing the entity-relationship model. She also helped with the normalizing procedure to make sure the database adheres to third normal form.

- Alex Vilamitjana concentrated on the relational model. He had to choose the right relationship's representation and translate the conceptual model into a relational structure database.

- The primary responsibility for the report and questions fell to Oriol Pros. This included writing all report, validating the accuracy and effectiveness of the models, and designing the project questions.

All team members actively participated in the discussion and approval of design choices notwithstanding this division of labour. It is also important to mention that all members ended up participating in all tasks to do in this phase.

1.2 Dataset description

The dataset selected for this project is based on the Olympic Games and contains information about athletes, events, and results for the edition of the Games of 2024. It is provided in CSV (Comma-Separated Values) format and is composed of several files, each representing different aspects of the Olympic competitions.

Firstly, the athletes.csv, coaches.csv, and technical_officials.csv files are made of the personal data of all accredited participants, which is athletes, coaching staff, and officiating personnel respectively, and each of them is identified by a unique code and linked to a delegation via NOC code. Then the NOC.csv file provides the reference table mapping each three-letter code to its corresponding country name. Also, venues.csv lists the competition venues, events.csv catalogues every competition event by sport, gender category, and venue, and schedules.csv details the timetable at the phase level, which shows qualifications, semifinals, finals with date and time information. Finally, medals.csv records the medal sets awarded per event, and medallists.csv attributes each medal to the individual athlete and delegation that won it.

The dataset presents three design-relevant characteristics. Athletes, coaches, and technical officials share some personal attributes despite being stored in separate files. The medallists.csv file duplicates data already present elsewhere in the dataset, such as athlete identity, delegation, and event. The NOC.csv file is limited to a code-to-name mapping, containing no geographic or linguistic data.

Even though the dataset is large, it is possible to transform it into a database, while relating each file to another, and make it as clear as possible.

1.3 Validation questions

1. Which athletes have won more medals than the average number of medals won per athlete across all disciplines?
2. Which countries have managed to win medals in all three categories for individual athletes: gold, silver, and bronze?
3. Which coaches are currently managing more athletes than the average number of athletes per coach?
4. Which 5 disciplines have a higher number of female participants than male participants?
5. Which athletes have managed to win medals and participate in more than 10 unique events?
6. Which venues have hosted events belonging to more than one different sport?
7. Which athletes representing Spain have also studied in University at Spain?
8.Which coaches are training athletes across more than one discipline at the same time?
9. Which countries have sent more athletes to compete than the average number of athletes per country?
10. Which sports have had their events spread across the highest number of different venues?
11. Which athletes share the same personal hero (father) and have also won at least one medal?
12. Which disciplines include both athletes born in the same country they represent and athletes born in a different country from the one they represent?
13. Which athletes were born in the same country they are representing and have won at least one gold medal?
14. Which disciplines have an average athlete age that is higher than the global average age across all disciplines?
15. Which organizations have technical officials involucrate in more than ten different disciplines?
16. Which athletes have won at least one medal and also have a sister who also does sports?
17. Which countries have athletes competing in every discipline currently scheduled in the Olympic Games?
18. Which coaches have never had any of their athletes win a medal throughout the competition?
19. Which nationalities appear most frequently among the athletes who have won gold medals?
20. Which disciplines have the lowest total number of athletes registered to compete?
21. Which 3 days in the schedule have the highest amount of scheduled events starting at the same time?
22.Which athletes are currently living in a country different from the one they are officially representing?
23.Which sports have had all their events concentrated in the fewest number of venues?
24.Which venues have hosted the highest number of different events within a single competition day?
25.Which countries have the largest number of athletes competing but have not won any medals so far?
26. Which countries have won more team gold medals than the combined total of their team silver and bronze medals?
27. Which countries have both individual medalists and team medalists?
28. Which sport is the most trained among athletes that have won a medal?
29.Which venues have had at least one scheduled event on both the first and the last day of the Olympics

30. Which are the top 3 countries with the highest number of athlete medalists whose training sessions have an intensity level above 7?

2 Entity-relationship model

2.1 Diagram

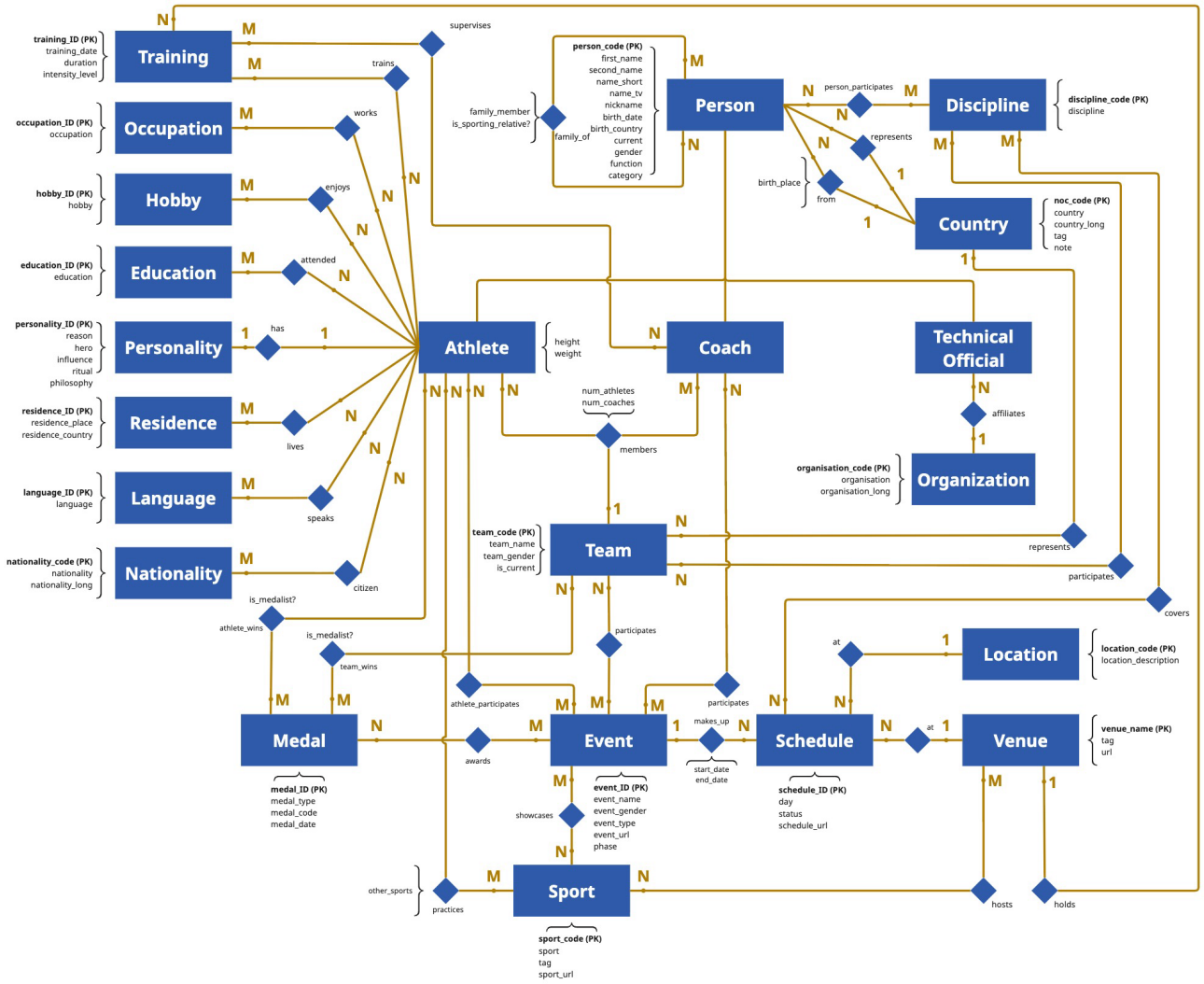


Figure 1: ERM in miro. Link: https://miro.com/app/board/uXjVGylzEzU=/?share_link_id=28893936811

2.2 Justification (4-6 pages)

The project's Entity-Relationship (ER) model was created to fully describe the data produced by the 2024 Olympic Games in Paris. The model's main goal is to capture, organize, and connect the three primary domains of the Games: the competitors, which include athletes, coaches, and technical officials; the competition infrastructure, which includes sports, events, schedules, venues, and locations; and the outcomes, which are focused on medals and medallists. It is crucial that its design guarantees data integrity and the ability to respond to a variety of questions by modeling these domains in a logical and normalized relational model.

Once the members decided on what dataset to work on, a main idea emerged: there would be an entity per each *csv* file available. Thus, so far there were the entities *athletes, coaches, technical officials, teams, medallists, medals, events, schedules, venues and NOCs*. From here, the model

would be developed, changing and arranging new entities and attributes according to the needs of those specific moments. The following paragraphs describe the thought process of the whole conceptual model based on these main entities initially created.

Person Entity

Firstly, a member started working on the athlete, coach and technical-officials dataset, thus the *Athlete, Coach and Technical-official* entities. It soon realized that among these three files there were a lot of attributes in common, so it was decided to make a generalization of *Person*, which would have all common attributes these files shared. We soon realized that there were too many columns to be considered as attributes. Due to normalization forms, which is required in the project statement, it took no time to realize that some attributes didn't directly depend on the person's primary key, thus a new entity had to be created so that all its attributes do depend ok the PK. The following entity was created because of this reason:

- *Country*: Again, the same scenario was repeated with this entity. The athlete's dataset had some columns such as *country_long that* did not depend on the person's code, and that's why it was decided to make it a new entity. Like this, it was ensured that the model was normalized, and for those people who didn't have other attributes relation to *country* (such as for coaches and technical officials), it simply would mark them as null. Thus, this entity had also a strong relation to the *NOC*'s CSV file, which created a mapping of countries, so that it would be easier to respect normalization among the model. This file was very useful, since it had all columns most people's entity also had, creating a direct mapping without need or creating new primary keys. Like so, the country's PK was its code, since it is unique, and it is a N to 1 relationship since each person represented 1 country, but a country is represented by multiple people. Also, we added another N to 1 relationship that included the birth information of an athlete, since it was clearly related to country.

Also, regarding the person's entity, two extra relationships were created. Firstly, the relationship with a new *discipline* entity, since if we included discipline as a person's attributes, it wouldn't be following atomicity, which ensures each column has only 1 value. This meant that a person could be enrolled with more than 1disciplines, thus this new entity and relationship were created, following an N to M relationship, since a person could be implicated in more than one discipline, and a discipline could be taken part by multiple people. Secondly, the reflexive relationship within itself, due to the presence of the column *family* in the athlete's CSV file. What this is indicating is that a person (in this case an athlete), if it has some family members, they're also under the people umbrella. A flag as also been included to distinguish whether this family member is also known as an athlete or sporting relative.

By making this generalization of person, coach and technical official ended up with no attributes, since most of them were in common with each other. It's important to mention that if an attribute was shared among 2 out of 3 entities, the attribute would be moved then to the generalized entity (person), and that's why they have no singular attributes.

Athlete Entity

Once the generalization had been made, a lot of attributes present in the athlete's CSV file were moved to it, leaving athlete with less attributes than before. However, there were some that still didn't fulfill the normalization requirement, then, some new entities were created according to it:

- *Occupation*: This entity was created because some athletes could have more than one occupation, thus not respecting atomicity. Like this, a new ID for occupation was created to uniquely identify each occupation type and then linked to athlete following an N to M relationship, since an athlete could have multiple occupations and an occupation could be done by multiple athletes.

- *Hobby*: Same issue happened with *Hobby*, since some athletes had more than one hobby. Then a new ID was created for each hobby, and it was linked with an N to M relationship with athlete, since an athlete could enjoy multiple hobbies and a single hobby could be enjoyed by multiple athletes at the same time.

- *Education*: Again, same thing happened with the *Education* entity, since there were some athletes who presented multiple education entities which he/she had attended. Thus, a new ID was created per each education type, and it was related to athlete with an N to M relationship, since an athlete could have attended to multiple educational sites, and an educational site could have had been attended by multiple athletes.

- *Language*: Once again, multiple athletes had more than 1 single value in the language column, so this entity was created, with a new ID representing each language. Then, it was linked with athlete by an N to M relationship, since an athlete could know how to speak multiple languages at the same time, and a single language could be spoken by multiple athletes.

- *Nationality*: Finally, following the same example as *Language*, it also happened with nationality, since it could be that some athlete had more than one nationality. That's why this entity was created and related to athlete by an N to M relationship, since an athlete could have more than one nationality and a nationality has more than one person.

It is also important to mention that entities such as *Nationality* and *Language* were not related to *Country*, even though in real life they present a clear relationship. This is because if so, a huge number of unnecessary relationships to represent this dataset would be made, overcomplicating the model, and making it difficult to code afterwards. Thus, any relationship that had nothing to do with the CSV files, was completely ignored, even though in real life a relationship among the two entities would have made sense.

Also, for the *Athlete* entity, some attributes were grouped into a new entity, to avoid redundancy and always ensure normalization. Firstly, *Residence* was created to store all attributes that depended on the attribute residence, which was made primary key for this new entity. Like this, we ensured that all remaining attributes in Athlete, depended on its primary key. It was later related to *Athlete* with an N to M relationship, since an athlete could have multiple residences and a residence could be occupied by multiple athletes. Also, the entity *Personality* was created, to group some attributes that had to do with the athlete's personality. Since its primary key had to be unique, it was considered that

the attribute reason was the best fit, since the reason for each athlete to become who they want to become is always unique, unrepeatable. This Personality entity had a 1 to 1 relationship, since an athlete has a unique personality, which is impossible to be repeated.

Coach Entity

For the *Coach* entity, all its attributes were incorporated into the generalized *Person* entity, leaving it without any specific attributes of its own. However, it was still retained in the model to define certain relationships, which are explained later in this report.

Technical Official Entity

The *Technical-Official* entity was restructured after all its attributes were incorporated into the generalized Person entity. It is now used to define a relationship with a new entity called *Organization*, based on the CSV data indicating that each technical official is affiliated with one organization. The *Organization* entity includes all attributes that depend on its primary key (the organization code). Keeping these attributes within the *Technical-Official* entity would violate normalization principles. The relationship between *Technical-Official* and *Organization* is N to 1, since each technical official can be affiliated with only one organization, while each organization can be associated with multiple technical officials.

Team Entity

The *Team* entity was derived directly from the teams CSV file. It retains its core attributes, which are team code (PK), team name, team gender, and status, since all of them depend exclusively on the team's primary key, respecting normalization. Beyond its attributes, the most important aspect of this entity is the ternary relationship it holds with *Athlete* and *Coach*. This design choice was made because the relationship between a coach and an athlete is inherently context-dependent on the team they belong to. For example, a coach may train the same athlete under different team contexts, or an athlete could be associated with different coaches depending on which team is considered. Since the meaning of this relationship cannot be captured accurately with binary associations alone, a ternary relationship was the appropriate modeling decision. The cardinality of this relationship is M to N between athletes and coaches, mediated through the team. Additionally, *Team* participates in further relationships with *Event* and *Organization*, which are described in their respective sections below.

Event Entity

The *Event* entity was created from the events CSV file and holds the attributes that directly depend on its primary key (*event_ID*): *event name*, *event gender, event type, event URL, and phase*. These attributes were kept within the entity since they all directly depend on the PK, fully satisfying normalization requirements. The *Event* entity sits at the center of several key relationships in the model. First, it has an N to M relationship with *Team*, since multiple teams can participate in a given event, and a team can compete in multiple events. Second, it has an N to M relationship with *Medal*, since an event can award multiple medals, and a medal is always associated with a specific event. Third, *Event* is related to *Sport* through an N to M relationship, since a sport can

encompass multiple events and an event always belongs to a sport. Finally, *Event* connects to *Schedule*, since an event is made up of multiple scheduled sessions, forming a 1 to N relationship where one event can have multiple schedule entries, but each schedule entry belongs to a single event.

Venue Entity

The *Venue* entity was derived directly from the venues CSV file and holds the attributes that depend exclusively on its primary key (*venue_ID): venue name, tag, and URL*. Since all attributes depend solely on the PK, the entity fully satisfies normalization requirements without the need to extract any further sub-entities. The *Venue* entity plays an important role in anchoring the physical dimension of the Games within the model. It maintains a 1 to N relationship with *Schedule*, since a single venue can host multiple scheduled sessions throughout the Games, but each schedule entry takes place at only one venue. Additionally, *Venue* is related to *Location* through a M to 1 relationship, since multiple venues can share the same broader location, but each venue belongs to only one location. The *Location* entity itself was created precisely to avoid storing location descriptions redundantly across multiple venue records, ensuring that the spatial context of each competition site is stored once and referenced consistently. Finally, *Venue* also maintains a relationship with *Sport* through the hosts relationship, since a venue can host multiple sports and a sport can be hosted across multiple venues, forming an N to M cardinality.

Schedules Entity

The *Schedule* entity was derived from the schedules CSV file and captures temporal and logistical information about each planned session of the Olympic Games. Its attributes, which are *schedule_ID* (PK), *day, status, and schedule URL*, all directly depend on the primary key, satisfying normalization principles. The *Schedule* entity has a central role in linking the competition structure to its physical locations. It maintains an N to 1 relationship with *Venue*, since multiple schedule entries can take place at a single venue, but each scheduled session occurs at only one venue at a time. Additionally, it holds a relationship with *Event*, as described above, where each schedule entry corresponds to one event, but one event can span multiple scheduled sessions. The inclusion of this entity was important to allow the model to represent the temporal dimension of the Games, when and where each session occurred, without overloading the *Event* entity with redundant or non-dependent attributes.

Medal Entity

The *Medal* entity was derived from the medals CSV file and stores information about each medal awarded during the Games. Its attributes are *medal_ID (PK), medal type, medal code, and medal date*, and all directly depend on the primary key, ensuring full compliance with normalization requirements. An important design decision related to this entity was the creation of the separate *Medallist* relationship. Rather than including athlete information directly within the *Medal* entity, a distinct relationship was modeled to connect *Medal* with *Athlete*, since a medal can be won by multiple athletes (in team events) and an athlete can win multiple medals across different events. This separation avoids data redundancy and keeps the *Medal* entity focused solely on the medal itself. Furthermore, *Medal* maintains an N to M relationship with *Event*, since each

event can produce multiple medals, and medals are awarded across multiple events throughout the Games.

Also, a deliberate decision in the construction of this ER model was the exclusion of a dedicated *Medallist* entity, despite the existence of a medallists CSV file in the original dataset. After carefully analyzing its columns, it became clear that every attribute it contained was already represented elsewhere in the model. Information such as the athlete's identity is stored in the *Person* and *Athlete* entities, medal details are held within the *Medal* entity, the event context is covered by the *Event* entity, and country representation is modeled through the *Person–Country* relationship.

Introducing a standalone *Medallist* entity would therefore have introduced direct data redundancy, violating the normalization principles that guided the entire design. Instead, the information the medallists CSV implicitly conveys, which athlete won which medal in which event, is naturally expressed through the existing relationships between *Athlete*, *Medal*, and *Event*.

Training Entity

Finally, to finish explaining the model, a new entity created from zero has been added, Training. This entity was introduced as the mandatory new CSV file that the team had to create. The decision behind choosing training sessions as a new entity is because with the three existing entities in the model, Athlete, Coach, and Venue, a training session can be created, performed by one athlete, supervised by one coach, and takes place at one venue, making it more realistic to later on following phases create dataset that can correlate better with the real world.

The Training entity holds the four attributes, all of which depend exclusively on its primary key, following normalization requirements. The *training_id* is the primary key of the entity, registering each training session with a unique value. The *training_date* records the day on which the training session took place. The *duration_min* stores the duration of the session expressed in minutes, a format chosen to facilitate data interpretation and allow greater flexibility when performing calculations. Finally, *intensity_level* is an indicator of how demanding the training session was, simulating an index called RPE (Rate of Perceived Exertion), which measures and tracks exercise intensity on a scale from 1 to 10 based on how hard the body feels it has worked during any given training session.

Regarding its relationships with other entities, Training follows a N to M relationship with Athlete, since each training session can have more than one athlete, and an athlete can have multiple training sessions across the preparation period. The same logic applies to Coach, since each session can be supervised by more one coach, and a coach can oversee many sessions throughout the same period. Finally, the relationship with Venue is N to 1, since a training session takes place at one specific venue at a time, but a venue can host many different training sessions across different days and athletes.

- **CSV**

The CSV file associated with this entity, named training_sessions.csv, was generated with the tool provided in the statement, *mockaroo*. It contains 72,232 records representing training sessions held during the three months prior to the Olympic Games, specifically from May 11, 2024, to August 11, 2024, (the date that the Olympic Games finished). This period was chosen because it represents the most intense preparation phase before the competition begins, making the data realistic for the following phases.

All the attributes from the CSV file are the ones from the *Training* entity, as well as *athlete_code*, *coach_code* and *discipline*. Including these extra attributes makes the file more coherent with the data overall, it defines the relations that the *Training* entity has and improves the readability of the CSV as a whole.

Since in the *mockaroo* page data can only be generated randomly, first a sample file was created with the corresponding basic attributes but organized with random entries, and afterwards the file was updated by Claude to link the file attributes directly with values from the existing CSV files.

Specifically, every *athlete_code* corresponds to a verified entry in athletes.csv and every *coach_code* corresponds to a verified entry in coaches.csv.

Regarding data volume, every athlete in the dataset has been assigned a minimum of 5 training sessions, distributed randomly across the date range. The sessions are ordered by date, with a random order within each day. It is also important to note that even though the CSV file includes the column *discipline* for readability purposes, this attribute will not be stored in the *Training* table in the database. This decision was made consciously to respect normalization principles, since this value is redundant and can always be retrieved through joins with the *Discipline* table.

All the details of the use of AI will be described down below in the corresponding part.

3 Relational model

3.1 Diagram

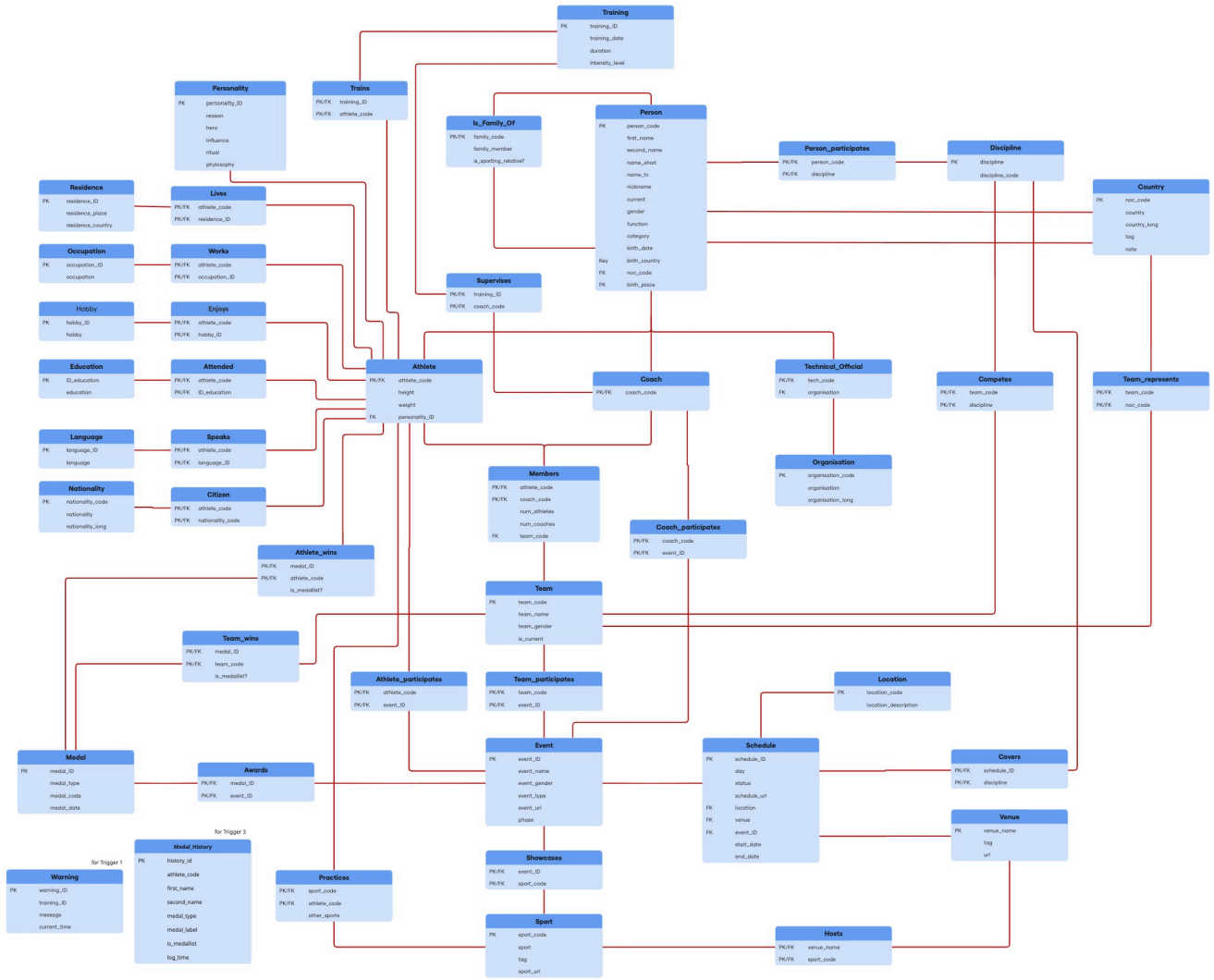


Figure 2: RM in miro. Link: https://miro.com/app/board/uXjVGubx3O8=/?share_link_id=55481093049

3.2 Justification (1-3 pages)

Once the ERM had been fully designed and implemented, each entity was systematically transformed into its corresponding relation, and the relationships connecting those entities were converted into the relational model according to their respective cardinalities. Main entities such as *Person*, *Event*, *Sport* and *Country*, which serve as central entities interconnected with a large number of other entities throughout the schema, were directly mapped into individual tables, each populated with their corresponding attributes. In doing so, their primary keys were preserved, thereby guaranteeing entity integrity across the entire model.

Relationships

When it comes to 1:1 and 1:N relationships, these were implemented by introducing a foreign key from one of the participating entities into the table on the N-side of the relationship. In the particular

case of a 1:1 relationship, the placement of the foreign key was decided based on whichever side would prove most beneficial for subsequent query operations. A clear illustration of this is the relationship between *Athlete* and *Personality*, where the foreign key of *Personality* is placed inside the *Athlete* table, as this design choice allows for one fewer join condition to be required when querying the data.

Furthermore, in situations involving either 1: N or 1:1 relationship where attributes belong to the relationship itself rather than to either entity individually, those attributes are incorporated into the table on the N-side of the relationship. This is exemplified by the relationship between Schedule and Event, where the attributes *start_date* and *end_date* are included on the N side accordingly.

On the other hand, N:M cardinality relationships were handled differently: rather than assigning a foreign key to an existing table, an entirely new table is created to represent the relationship. This new table holds the primary keys of both connected entities, serving simultaneously as primary and foreign keys within that table. As was the case with the previously described relationship types, any attributes that belong to the relationship between the two entities are also housed within this newly created table. These junction tables are assigned names that are intentionally descriptive and reflective of the nature of the relationship they represent, for instance, the fact that an *Athlete* Participates in an *Event* is captured both semantically and structurally in the table name.

A particularly noteworthy relationship case is the ternary association involving *Athlete*, *Coach* and *Team*. Given that this relationship carries a N:M:1 cardinality, a new table named *Members* was created to encapsulate it. This table holds the primary keys of both *Athlete* and *Coach* as combined primary and foreign keys, and additionally contains all attributes associated with the relationship among the three entities. The foreign key of *Team* is also included within the *Members* table, as the cardinality of 1 on the *Team* side implies that both an athlete and a coach belong to exactly one team at a time. This structure ensures that the uniqueness of each record in the table is determined solely and specifically by the athlete-coach combination.

Generalization

As was established during the design of the ERM, *Person* acts as a generalization of athletes, coaches and technical officials. Within the ERM, the generalized entities do not carry their own primary keys, since the super-entity *Person* already holds the identifying attribute *person_code*. When transitioning to the relational model, however, the primary key of *Person* is propagated down to each sub-entity, where it takes on the dual role of both primary key and foreign key. Additionally, to clearly differentiate between the identifying attributes across the various sub-entities, distinct names were assigned to these composite keys, for example, the key inherited by the *Athlete* sub-entity is referred to as *athlete_code.*

4 Datatype selection (1-2 pages)

The physical model translates the relational schema into the actual Oracle SQL table definitions. At this stage, one of the main challenges was selecting appropriate data types for each attribute, since the design needed to accommodate all possible values present in the CSV files while also ensuring consistency, scalability, and data persistence for the later phases of the project.

Throughout the schema, two distinct strategies are used to identify records: numeric surrogate keys and alphanumeric natural keys derived from the source data.

- **NUMBER(10)** is used for all surrogate primary keys that we generated artificially, so there was no specific ID in the CSVs but based on our Relational Model needed a determined ID. For instance, attributes such as *person_code, event_ID, medal_ID, schedule_ID, personality_ID, training_ID, occupation_ID, hobby_ID, education_ID, residence_ID, and language_ID*. A 10-digit integer provides a sufficiently large range to accommodate the full Olympic dataset and any realistic extensions.

- **VARCHAR2(3)** is used for all three-letter natural codes that come directly from the original CSV files, such *as noc_code, sport_code, venue_code, location_code,* and *discipline_code*. These are fixed-length identifiers standardized by the Olympic Committee (found out after looking on the internet) and a three-character variable-length string is the minimum safe choice. VARCHAR2 is how Oracle works with strings. Also, we chose it over CHAR because Oracle pads CHAR values with spaces, which can cause unexpected mismatches in join and comparison operations, as we read on Stack Overflow.

- **VARCHAR2(4)** is used for *organisation_code* in the Organization table, since the source data contains organization codes that can reach four characters, as verified in the TECHNICAL_OFFICIALS CSV file.

- **VARCHAR2(25)** is used for *team_code*, since team identifiers in the dataset are longer and more varied than three-letter NOC codes, sometimes including gender or sport suffixes, thus the safest choice was a longer string to ensure that any kind of code inserted would be valid.

On the other hand, a deliberate distinction is made between short, bounded strings and potentially long textual content, based on some answers we've seen on the CSV files.

- **VARCHAR2(255)** is the standard choice for names and descriptive labels with a, such as *country, venue_name, sport, discipline, event_name, team_name, first_name, second_name, occupation, language, nationality, residence_place, birth_place, birth_country,* and similar fields. We believe the 255-character limit covers all realistic values in the dataset, as we've tried to reassure by looking in the CSV files. Oracle stores VARCHAR2 values in variable length, so shorter entries consume less space accordingly, thus, there's no problem if some entries are shorter, since the important part is that all entries are valid.

- **VARCHAR2(500)** is applied to nickname in the Person table, since athlete nicknames in the source data can be compound or notably long, occasionally exceeding the standard 255-character threshold. The reason we chose to make it a “string” and not a CLOB (explained afterwards) is because at some point in the inserts it was easier to compare strings rather that variables interpreted as a text.

- **VARCHAR2(800)** is used for the influence attribute in the Personality table, as this field contains free-text biographical descriptions of the people who have influenced each athlete, which can be longer than a standard name but shorter than a full narrative. The reason on why we chose the type string and not CLOB is the same mentioned previously.

- **CLOB** (Character Large Object) is reserved for fields that contain long, unstructured, or unpredictably sized text: reason, hero, ritual, and philosophy in Personality, as well as hobby in Hobby, education in Education, *family_member* in *Is_Family_Of*, and *other_sports* in Practices. These fields originate from open-ended survey-style responses in the athletes CSV, where the length can vary dramatically between entries. The tradeoff is that CLOB values cannot be directly indexed and are slightly less efficient in join operations, so we had some issues when inserting data, and that’s why some attributes that might be correctly considered as CLOB, we ended up changing its type to a string, for a more practical use.

Also, **VARCHAR2(255)** is used for all URL fields, including *sport_url, url* in Venue, *event_url, schedule_url,* and the *url* field in Schedule. While modern URLs can theoretically be longer, all URLs present in the Olympic dataset are well within this limit.

Several attributes represent a small, closed set of possible values and are modelled with short VARCHAR2 types combined with CHECK constraints where appropriate.

- **VARCHAR2(1)** is used for single-character categorical codes: *event_gender* and gender in Person (e.g. 'M', 'F', 'X'), as well as category in Person (used to distinguish athlete categories). Single-character storage is maximally compact for these well-defined enumerations.

- **VARCHAR2(5)** is used for boolean-like flags stored as string literals, specifically *is_current* (constrained to 'True' or 'False') in Person and Team, *is_medallist* in *Athlete_wins* and *Team_wins*, and *is_sporting_relative* in *Is_Family_Of*. Oracle's SQL dialect lacks a native BOOLEAN type for table columns, so the standard workaround is a short VARCHAR2 with a CHECK constraint that enforces the domain. The value 'False' requires five characters, which determines the minimum length.

- **VARCHAR2(25)** is used for status in Schedule, *event_type* in Event, *medal_type* in Medal, function in Person, and *residence_country* in Residence. These fields contain short categorical descriptors that exceed a single character but are still bounded in length.

Additionally, for numeric measurement attributes in the Athletes, Training sessions, and Medals CSV files:

- **NUMBER(5,2)** is used for height and weight. This allows values up to 999.99, which comfortably covers athlete heights in centimeters and weights in kilograms, while limiting precision to two decimal places as appropriate for these physical measurements.

- **NUMBER(10)** is used for *medal_code* in Medal, duration and *intensity_level* in Training, and *num_athletes* and *num_coaches* in Members. These are integer-valued quantities with no fractional component. *medal_code* is a numeric reference code from the source data; duration records session length in minutes as a whole number; *intensity_level* stores the RPE index (1–10) as an integer; and *num_athletes* / *num_coaches* are straightforward counts.

Moreover, **DATE** is used for *birth_date* in Person, *medal_date* in Medal, and day in Schedule. Oracle's DATE type stores both date and time components with second-level precision, though for these fields only the date portion is relevant. Using DATE rather than VARCHAR2 was a topic of discussion within the members, since we had been told using dates might be problematic in Oracle. However, for these attributes we decided to keep the date structure since it seemed that there were no problems when inserting the data.

Nevertheless, **VARCHAR2(25)** is used for *start_date* and *end_date* in Schedule, and *training_date* in Training. This was a pragmatic decision driven by the format of the source data: these fields contain datetime strings with timezone information (for example, "2024-07-26T09:00:00+02:00") that do not map cleanly to Oracle's TIMESTAMP WITH TIME ZONE type without a conversion function. Storing them as strings preserves the original format and avoids data loss during loading, with the understanding that any time-based computation would require explicit TO_TIMESTAMP conversion in queries.

Finally, **VARCHAR2(255)** is used for all tag fields present in Country, Sport, Venue, and related tables. Tags are short textual labels used for categorization or display purposes in the original dataset, and their length is bounded in practice. No special type is required.

In general, the datatype choices across the schema follow two guiding principles. First, compactness: fixed-domain values such as codes and flags use the smallest VARCHAR2 length that safely covers all observed values. Second, correctness: dates are stored as DATE to enable temporal operations; large free-text fields use CLOB to avoid silent truncation. Together, these decisions ensure that the physical model will not give any trouble when dealing with real data.

5 Physical model codification (1-2 pages)

The physical model was implemented as a single Oracle SQL script that translates the full relational schema into executable table definitions, data loading instructions, and validation queries. The code follows a structured sequence designed to respect referential integrity throughout, since some tables depend on other ones.

The code starts with a set of DROP TABLE statements covering every table in the diagram. These are ordered in reverse dependency, dropping relationship tables before the entity tables they reference, so that foreign key constraints do not block the drops. This makes the code easier to modify if any unexpected change happens, it can be re-run from scratch at any point without cleaning up manually, which was during the development and testing of the model was essential.

The CREATE TABLE statements are written after, ordered from the most independent entities to the most dependent ones, so that tables with no foreign keys, such as Country, Sport, Discipline, Venue, Location, and Organization, as well as the auxiliary attribute tables Personality, Occupation, Hobby, Education, Residence, Language, and Nationality, are defined first. Then, tables such as Event, Medal, Team, Training, and Person come next, since they reference the foundational tables already created. Finally, the tables Athlete, Coach, and Technical_Official, grouped with all relationship tables, are created last, since every table they depend on is already in created, making the order of creating tables in the good line.

Each CREATE TABLE statement has its own columns with its chosen datatype, there are NOT NULL constraints that are all the PRIMARY KEY, and there are all the FOREIGN KEY references to previously defined tables. Where an attribute is a Boolean type, since in oracle Boolean doesn’t exist, an inline CHECK constraint is added directly within the column definition to. This applies to the *is_current* flag in Person and Team, which is constrained to the literals 'True' and 'False', the *is_medallist* flag in *Athlete_wins* and *Team_wins,* and the *is_sporting_relative* flag in *Is_Family_Of.*

Composite primary keys are used throughout the relationship tables. In tables such as *Athlete_participates, Coach_participates, Team_represents, Hosts, Showcases, Covers*, and all other junction tables, the primary key is declared as a combination of the two or more foreign key columns that together uniquely identify each row. This design eliminates the need for artificial replacement keys in relationship tables.

Foreign key lines follow a specific pattern, every reference is placed after the column list, using FOREIGN KEY ... REFERENCES syntax that points to the primary key of the parent table. The ordering of CREATE TABLE statements makes that the referenced table always exists whenever the foreign key is created, avoiding dependency errors during code execution.

6 Data import (2-3 pages per file)

The data import process consists of a series of INSERT ... SELECT statements that extract data, transform it, and load it from the raw temporal tables into the normalized tables. Each temporal table mirrors one of the original CSV files and was loaded into Oracle prior to running the main code. The methodology applied to each file is described in detail below, covering the source columns used, any structural problems found, the transformation logic applied, and the join or deduplication strategy required to produce clean, constraint-compliant rows in each target table.

6.1 noc.csv

The NOC file is the most structurally simple of all source files and serves as the foundation for the entire import process. It contains one row per National Olympic Committee and has five columns: *code, country, country_long, tag*, and *note*. These map directly and without transformation to the five columns of the Country table, making this the only import in the entire script where no JOIN, no deduplication logic, and no column renaming were required.

```
INSERT INTO Country (noc_code, country, country_long, tag, note)
SELECT code, country, country_long, tag, note
FROM NOCS;
```

The Country table is created first among all tables in the script. Country is referenced as a foreign key by a large number of downstream tables, including Person (via both *noc_code* and *birth_country*), Team_represents, and Citizen. If Country was not created before any of these tables were loaded, the foreign key constraints would reject all incoming rows.

For other simpler inserts such as this one, the same code structure is followed.

6.2 venues.csv

The venues file presents an important structural problem, it contains descriptive metadata about competition venues, specifically each venue's tag and URL, but it does not include the venue codes that serve as the primary key of the Venue table. Venue codes are only present in the schedules file, which lists the code for each venue referenced by a schedule entry. This fragmentation between two source files meant that the Venue table could not be created from venues.csv alone.

```
INSERT INTO Venue (venue_code, venue_name, tag, url)
SELECT DISTINCT s.venue_code, s.venue, v.tag, v.url
FROM SCHEDULES s
LEFT JOIN VENUES v ON v.venue = s.venue
WHERE s.venue_code IS NOT NULL;
```

The solution was to use a LEFT JOIN between the SCHEDULES staging table and the VENUES staging table, matching on the venue name string, which is the only column shared between the two files. The *venue_code* and *venue_name* were drawn from SCHEDULES, while tag and url were drawn from VENUES. A LEFT JOIN rather than an INNER JOIN was deliberately chosen to preserve venue entries from SCHEDULES even in cases where the corresponding VENUES row might be absent, ensuring no venue code present in the schedule data is lost from the Venue table.

The main problem was that the join key between the two files was a free-text venue name string rather than a reliable surrogate key. During initial testing with an INNER JOIN, several venue codes appeared in the SCHEDULES staging table but returned no match in VENUES because of minor spelling discrepancies or trailing whitespace differences between the two files. Switching to a LEFT JOIN preserved all venue codes while allowing the *tag* and *url* fields to be NULL for unmatched venues. A second problem was the large number of duplicate combinations produced naturally by the join, since the same venue appears across hundreds of schedule rows. A DISTINCT clause was applied to collapse all duplicates before insertion. Validation confirmed that the count of distinct venue codes in the populated Venue table matched the count of distinct venue codes in SCHEDULES.

```
SELECT COUNT(DISTINCT venue_code)
FROM SCHEDULES
WHERE venue_code IS NOT NULL;

SELECT COUNT(DISTINCT venue_code)
FROM Venue;
```

This approach highlights a general design tension present throughout the import process: the raw CSV files were not designed with relational normalisation in mind, and information that logically belongs to a single entity is often distributed across multiple files without a shared surrogate key. The venue name string acts as a natural join key here, but its reliability depends on exact string consistency between the two files. Any spelling discrepancy between the venue name as written in SCHEDULES and as written in VENUES would result in a NULL tag and *url* for that venue. This risk was accepted as the only feasible approach given the available data.

The Venue table must be populated before Schedule, since Schedule contains a foreign key reference to *venue_code*. It must also be populated before Hosts, which links venues to sports. The validation query compares the count of distinct venue codes in SCHEDULES against the count in the populated Venue table to confirm no codes were dropped during the join.

6.3 events.csv

The events file catalogues every competition event by sport and discipline, and its role in the import process is somewhat indirect. Rather than being the primary source for the Event table, it is used primarily to populate the Sport table and the Showcases relationship table. This design decision was made because the Event table in the schema is keyed on a combination of event name, gender, and *event_type*, and only the schedules file contains all three of these attributes together in the same row.

For the Sport table, a SELECT DISTINCT on *sport_code*, *sport, tag*, and *sport_url* from the EVENTS staging table was sufficient, since each sport appears many times in the file but always with the same metadata. The *sport_code* serves as the natural primary key and requires no surrogate generation. Sport must be created early in the sequence because it is referenced by Hosts, Showcases, and Practices.

```
INSERT INTO Sport (sport_code, sport, tag, sport_url)
SELECT DISTINCT sport_code, sport, tag, sport_url
FROM EVENTS;
```

The events file also contributes to the Showcases relationship table, which records which sport each event belongs to. This was populated by joining the already-populated Event table to the EVENTS

staging table on event name, retrieving the *sport_code* for each matching event. The use of DISTINCT prevents duplicate pairs from entering what is a composite primary key table.

One important nuance in the use of this file is that the event names in EVENTS do not always include gender or *event_type* qualifiers, meaning that the join from Event to EVENTS on name alone can produce multiple matches for events that share the same name across genders or types. The DISTINCT clause handles this gracefully by collapsing any such duplicates before insertion, ensuring that Showcases contains only unique event-sport combinations regardless of how many raw matches the join produces.

6.4 schedules.csv

The schedules file is by far the most heavily used source in the entire import process, contributing data to five distinct tables: Discipline, Location, Event, Schedule, and Covers. Its central role reflects the fact that it is the most granular file in the dataset, with one row per scheduled session containing a full set of contextual attributes including sport, discipline, event name, gender, event type, venue code, location code, phase, status, day, start time, end time, and URL.

For the Discipline table, a SELECT DISTINCT on discipline and *discipline_code* from SCHEDULES was sufficient. Each discipline appears many times across schedule rows but always with the same code, so deduplication via DISTINCT collapses these to one row per discipline. Discipline uses the discipline name string as its primary key rather than the code, because the name is the value used in foreign key references throughout the schema, including Person_participates, Covers, and Competes.

For the Location table, the same SELECT DISTINCT approach was applied to *location_code* and *location_description*. Location codes are three-letter identifiers analogous to venue codes, and since location descriptions are stable across all rows sharing the same code, no aggregation was needed.

For the Event table, a more sophisticated approach was required. A logical event is uniquely identified by the combination of event name, gender, and *event_type*. In SCHEDULES the same logical event appears across many rows, one per scheduled session. The import groups by this triple and applies MAX to select a representative *url* and phase value:

For the Event table, a more sophisticated approach was required. A logical event is uniquely identified by the combination of event name, gender, and *event_type*. In SCHEDULES, the same logical event appears across many rows, one per scheduled session. The import therefore groups by this triple using GROUP BY and applies MAX to both *url* and phase to select a representative value for each group, since these attributes are consistent within a group and aggregation is simply a technical requirement of the GROUP BY clause. A ROW_NUMBER() window function ordered by event name, gender, and *event_type* generates a deterministic surrogate *event_ID*, ensuring that repeated executions of the script produce identical key assignments.

```
INSERT INTO Event (event_ID, event_name, event_gender, event_type, event_url, phase)
SELECT ROW_NUMBER() OVER (ORDER BY event, gender, event_type), event, gender,
event_type, MAX(url), MAX(phase)
FROM SCHEDULES
WHERE event IS NOT NULL
GROUP BY event, gender, event_type;
```

For the Schedule table itself, each row in SCHEDULES maps to one schedule entry, but the foreign key *event_ID* must be resolved before insertion. This is done by joining SCHEDULES to the already-populated Event table on the triple of event name, gender, and *event_type*. The *schedule_ID* is generated using ROW_NUMBER() ordered by *start_date* and event name, providing a stable row ordering. The location and venue columns store the three-letter codes directly from SCHEDULES, which are valid foreign keys to the already-created Location and Venue tables respectively.

Finally, the Covers relationship table, which links each schedule entry to its discipline, is populated by joining the Schedule table back to SCHEDULES on the combination of *url*, *location_code,* and *venue_code*. This three-column join key is necessary because no single column uniquely identifies a raw schedule row in a way that can be matched to the *schedule_ID* generated during the Schedule insert. The *url* alone would be ambiguous for entries that share a URL across disciplines, so the venue and location codes are added to make the match precise. DISTINCT is applied to prevent duplicate schedule-discipline pairs.

6.5 athletes.csv

The athletes file is the richest and most structurally complex source in the dataset. It is the sole contributor to a large number of tables and the primary contributor to several others. Its columns span personal identity, physical measurements, biographical free text, and denormalised list-valued fields for disciplines, events, languages, occupations, and family relationships. The import from this file therefore required the widest variety of transformation techniques used anywhere in the script.

The Person table receives its first batch of rows from this file. The columns *name_short*, *name_tv*, *nickname*, *is_current*, *gender*, *function*, *birth_date*, and *country_code* map directly to their Person equivalents. The *birth_country* column, however, required a LEFT JOIN to the Country table to convert the raw country name string stored in the athletes file into a valid *noc_code*, since *Person.birth_country* is a foreign key to *Country.noc_code* rather than a free-text field. A LEFT JOIN was used rather than an INNER JOIN to preserve athletes whose *birth_country* value does not match any entry in Country, avoiding data loss for athletes with non-standard or missing country strings.

A notable challenge was extracting first and second names from the single name column, which stores names in the Olympic format SURNAME Firstname. A regular expression was used to split the uppercase surname token from the rest:

```
INSERT INTO Person(person_code, second_name, first_name, name_short, name_tv,
nickname, is_current, gender, function, birth_date, noc_code, birth_place, birth_country)
SELECT a.code,
    TRIM(REGEXP_SUBSTR(a.name, '^(([A-Z]{2,}|de|del|von) )+')),
    TRIM(REGEXP_REPLACE(a.name,'^(([A-Z]{2,}|de|del|von) )+','')),
    a.name_short, a.name_tv, a.nickname, a."current",
    a.gender, a.function, a.birth_date, a.country_code,
    a.birth_place, c.noc_code
FROM ATHLETES a
LEFT JOIN Country c ON c.country_long = a.birth_country;
```

The Athlete table is populated with height, weight, and a resolved personality_ID. MAX aggregation on height and weight was applied per athlete code to handle any duplicate rows that may exist in the source. The personality_ID was resolved by joining to the already-populated Personality table on the influence column, using MIN to select a single personality record per athlete in cases of ambiguity.

The Personality table itself is sourced exclusively from the athletes file. The columns reason, hero, influence, ritual, and philosophy are extracted with DISTINCT and assigned surrogate keys via ROWNUM. The influence column was chosen as the join key when resolving *personality_ID* in the Athlete insert because it is the attribute most likely to be unique per personality record in practice.

The Residence table required a ROW_NUMBER() window function ordered by residence_place and residence_country to generate stable surrogate keys, since the combination of these two columns is what uniquely identifies a residence record. Filtering on residence_place IS NOT NULL excludes athletes for whom no residence information was provided.

For the Occupation and Language tables, the solution was to perform multiple INSERT passes using REGEXP_SUBSTR with incrementing positional parameters to extract one token per pass, then apply a DELETE to remove all but the lowest-ID duplicate after all passes had completed:

```
INSERT INTO Occupation
SELECT ROWNUM, TRIM(REGEXP_SUBSTR(occupation, '[^,]+', 1, 1))
FROM ATHLETES WHERE REGEXP_SUBSTR(occupation, '[^,]+', 1, 1) IS NOT NULL;

INSERT INTO Occupation
SELECT ROWNUM + 10000, TRIM(REGEXP_SUBSTR(occupation, '[^,]+', 1, 2))
FROM ATHLETES WHERE REGEXP_SUBSTR(occupation, '[^,]+', 1, 2) IS NOT NULL;

…

DELETE FROM Occupation o1
WHERE occupation_ID > (
   SELECT MIN(o2.occupation_ID) FROM Occupation o2
   WHERE o1.occupation = o2.occupation
);
```

The ROWNUM offset strategy (+10000, +20000, +30000) was introduced after an initial version without offsets caused ORA-00001 (unique constraint violated) primary key conflicts between passes, because ROWNUM restarts from 1 on each SELECT. Language required five passes instead of four since some athletes speak up to five languages.

Hobby and Education were each populated with a single INSERT using ROWNUM as the surrogate key, filtering on the non-null condition. Unlike Occupation and Language, these fields are stored as single values per athlete rather than lists, so no multi-pass extraction was needed. Both use CLOB as their datatype to accommodate the potentially long free-text values present in the source.

Nationality was populated with a SELECT DISTINCT on nationality_code, nationality, and nationality_long, filtered to exclude null nationality codes. This is straightforward since nationality codes are already three-letter identifiers comparable to NOC codes.

For the relationship tables sourced from the athletes file, several different join strategies were employed depending on how the relevant columns are structured in the source. Lives joins to Residence on the exact pair of residence_place and residence_country. Citizen inserts directly from the nationality_code column without any join, since the code is already the foreign key value. Enjoys and Attended both use exact string equality, matching the athlete's hobbies and education columns

respectively against the CLOB values stored in the Hobby and Education tables using TO_CHAR conversion.

Speaks uses REGEXP_LIKE with a comma-wrapped pattern to match each athlete's lang string against individual language values. Wrapping both the source string and the match value with commas, as in matching against ',' || l.language || ',', prevents partial matches where one language name is a substring of another.

Works uses a similar INSTR-based comma-wrapping approach applied to the occupation column in lowercase, ensuring that case differences between the athlete's raw occupation string and the cleaned Occupation table values do not cause missed matches.

Athlete_participates uses INSTR to check whether a clean event name from the Event table appears anywhere within the athlete's denormalised events string. This approach is necessary because the events column in the athletes file stores multiple event names concatenated in a single string without a consistent delimiter. The same INSTR strategy is applied to populate Practices, matching sport names from the Sport table against the athlete's disciplines string.

Person_participates for athletes uses REGEXP_LIKE to match discipline names enclosed in single quotes within the bracket-quoted list format used in the athletes disciplines column. This format, where discipline names appear as 'Discipline Name' within a Python-style list string, required a pattern that wraps the discipline name with escaped single-quote characters to avoid partial matches.

Another problem was the REGEXP_LIKE pattern used to populate Person_participates for athletes. The discipline names are enclosed in single quotes inside a Python-style list string, requiring the match pattern to wrap the discipline name with escaped single-quote characters to avoid partial matches:

```
INSERT INTO Person_participates (person_code, discipline)
SELECT DISTINCT a.code, d.discipline
FROM ATHLETES a
JOIN Discipline d ON REGEXP_LIKE(a.disciplines, "" || d.discipline || "")
WHERE a.disciplines IS NOT NULL;
```

Without the single-quote wrapping, a discipline name such as "Athletics" would incorrectly match rows containing "Athletics Jumps" or "Athletics Sprints".

Is_Family_Of is populated in two passes. The first targets athletes who have both a family column and a sporting_relatives column populated, assigning is_sporting_relative the value 'True'. The second pass targets athletes who have a family column but no sporting_relatives value, assigning 'False'. This conditional split was the cleanest way to encode the boolean flag directly from the source data without requiring a CASE expression.

6.6 coaches.csv

The coaches file contributed data to the Person table, the Coach table, and two relationship tables: Person_participates and Coach_participates. Its column set is narrower than the athletes file, containing no biographical free text and no list-valued fields for languages, occupations, or hobbies. This makes the coach import considerably simpler than the athlete import, though several important differences in column availability needed to be handled carefully.

The Person table receives coach records in a second INSERT pass that follows the athlete pass. Only the columns available in the coaches file are populated: person_code from code, is_current, gender, function, category, birth_date, and noc_code from country_code. The name columns first_name, second_name, name_short, name_tv, and nickname, as well as birth_place and birth_country, are all left as NULL for coach records because the coaches file does not contain this information. This is an accepted data limitation and does not compromise the integrity of the schema, since NULL is permitted for all of these columns in Person.

Notably, the coaches file does include a category column that is absent from the athletes file. This single-character field distinguishes different functional categories of coaching staff and is stored in Person.category. Since athletes do not have a category value, this column will always be NULL for athlete rows, while it is populated only for coach and technical official rows.

The Coach table itself contains only the coach_code column beyond its inherited primary key, reflecting the fact that all coach-specific data was absorbed into the generalised Person table during the ERM-to-relational translation. The insert is therefore a simple SELECT of the code column from COACHES with a direct one-to-one correspondence between source rows and target rows.

For Person_participates, coaches presented a different challenge compared to athletes. While the athletes disciplines column stores a bracket-quoted list of multiple discipline names, the coaches file stores disciplines as a single trimmed string value. This meant that the REGEXP_LIKE list-matching approach used for athletes was not applicable here. Instead, a direct equality join between the trimmed coaches disciplines value and the Discipline table's discipline column was used, which is both simpler and more reliable given the single-valued nature of the source column.

A problem we had was with Person_participates for coaches. Unlike the athletes file, which stores a bracket-quoted list of discipline names, the coaches file stores a single plain string value in the disciplines column. The REGEXP_LIKE pattern used successfully for athletes produced zero rows when applied to coaches because the plain string format contains no enclosing single quotes. The solution was to switch to a direct equality join for coaches:

```
INSERT INTO Person_participates (person_code, discipline)
SELECT DISTINCT c.code, d.discipline
FROM COACHES c
JOIN Discipline d ON d.discipline = TRIM(c.disciplines)
WHERE c.disciplines IS NOT NULL;
```

Applying TRIM was necessary because the raw values in the coaches disciplines column occasionally contained leading or trailing whitespace that prevented exact string equality matches. A third problem was Coach_participates, where the events column in the coaches file stores a long concatenated string of event names without a consistent delimiter. The INSTR-based approach was used, matching each event name from the Event table against the raw events string. This required that event names in Event were clean enough to serve as reliable substrings, which was confirmed by spot-checking several coach records manually.

6.7 technical_officials.csv

The technical officials file contributed data to three tables: Person, Technical_Official, and Organization, as well as to Person_participates. It is structurally similar to the coaches file in that it contains a category column and a disciplines field but lacks biographical free text. It differs

importantly in that it includes organisation affiliation data, which is not present in either the athletes or coaches files.

The Person table receives technical official records in a third INSERT pass following athletes and coaches. The columns populated are person_code from code, is_current, gender, function, category, and birth_date. Crucially, no noc_code is populated for technical official records, since the technical officials file does not include a country_code column. This reflects the real-world situation where technical officials are appointed by international federations rather than national delegations and are therefore not linked to a specific country in the dataset. The birth_country foreign key is similarly left as NULL for all technical official records.

The Organization table is populated exclusively from this file, since organisation affiliation data does not appear in any other source. The staging table TECHNICAL_OFFICIALS contains the columns organisation_code, organisation, and organisation_long, which map directly to the Organization table columns. A SELECT DISTINCT ensures one row per organisation code, since the same organisation is referenced by multiple technical officials. The WHERE clause filters out rows where organisation_code is NULL, avoiding an attempted insert of a null primary key. Organization must be created and populated before Technical_Official, since the latter references organisation_code as a foreign key.

The Technical_Official table stores only tech_code and the organisation foreign key. The organisation value is taken directly from the organisation_code column in the staging table, without any join, since the code is already the foreign key value. This is a simple one-to-one row correspondence between TECHNICAL_OFFICIALS and Technical_Official.

Person_participates for technical officials uses the same REGEXP_LIKE bracket-quoted list pattern applied for athletes, since the technical officials disciplines column uses the same multi-value list format. This is the one structural similarity between the technical officials and athletes files that required shared handling. DISTINCT prevents duplicate person-discipline pairs, and the insert runs as a third pass into Person_participates following the athlete and coach passes.

6.8 medals.csv

The medals file records every medal awarded during the Games, with one row per medal. Its columns are event, medal_type, medal_code, medal_date, and country, among others. The import from this file contributed directly to the Medal table and indirectly to the Awards relationship table.

For the Medal table, a ROW_NUMBER() window function ordered by the event column generates the surrogate medal_ID. The medal_type, medal_code, and medal_date columns are taken directly from the staging table MEDALS without transformation, with medal_date stored as a DATE type since its source values are valid Oracle date strings. No DISTINCT or deduplication was applied here, since each row in the medals file represents a unique medal award and duplicates are not expected in the source.

```
INSERT INTO Medal (medal_ID, medal_type, medal_code, medal_date)
SELECT ROW_NUMBER() OVER (ORDER BY event),
        medal_type, medal_code, medal_date
FROM MEDALS;
```

The Awards relationship table links each medal to the event in which it was awarded. Because the Medal table does not store the event directly (event information belongs to the Event table in the

normalised schema), Awards requires a multi-table join to resolve the correct medal_ID for each event-medal_type combination. The join chain passes through the MEDALLISTS staging table to obtain event context, through Event to resolve the event_ID, through MEDALS to match the raw medal record, and finally through Medal to obtain the surrogate medal_ID. A MIN aggregation on medal_ID per event_ID and medal_type combination ensures that exactly one medal_ID is assigned per event-medal_type pair, since the window-function-generated IDs may produce multiple candidates in edge cases. This join chain is one of the most complex in the entire script and reflects the degree to which the medals and medallists files were designed as complementary rather than self-contained sources.

```
INSERT INTO Awards (medal_ID, event_ID)
SELECT MIN(m.medal_ID), e.event_ID
FROM MEDALLISTS ml
JOIN Event  e  ON e.event_name = ml.event AND e.event_type = ml.event_type
JOIN MEDALS md ON md.event = ml.event AND md.medal_type = ml.medal_type
JOIN Medal  m  ON m.medal_type = md.medal_type
        AND m.medal_code = md.medal_code
        AND m.medal_date = md.medal_date
WHERE ml.medal_code IS NOT NULL
GROUP BY e.event_ID, ml.medal_type;
```

6.9 medallists.csv

The medallists file was not used to populate any standalone entity table. Every piece of information it contains is already represented in the schema through the athletes, medals, and events tables combined. Athlete identity is stored in Person and Athlete, medal details are held in Medal, event context is covered by Event, and country representation is modelled through Person. Creating a separate Medallist entity would have introduced direct redundancy, and the decision to omit it was described in the ERM justification.

Instead, the medallists file was used exclusively as a bridge source to populate the two win relationship tables: Athlete_wins and Team_wins.

For Athlete_wins, the join chain starts from MEDALLISTS, joins to Event on event name and event_type, then to Awards on event_ID, and finally to Medal on medal_ID and medal_type. The filter condition code_athlete IS NOT NULL restricts the insert to individual athlete medal records, excluding team entries. DISTINCT prevents duplicate medal-athlete pairs.

```
INSERT INTO Athlete_wins (medal_ID, athlete_code, is_medallist)
SELECT DISTINCT m.medal_ID, ml.code_athlete, ml.is_medallist
FROM MEDALLISTS ml
JOIN Event  e  ON e.event_name = ml.event AND e.event_type = ml.event_type
JOIN Awards aw ON aw.event_ID  = e.event_ID
JOIN Medal  m  ON m.medal_ID   = aw.medal_ID AND m.medal_type = ml.medal_type
WHERE ml.code_athlete IS NOT NULL;
```

For Team_wins, the same join chain is used but filtered on code_team IS NOT NULL. Because multiple medallists rows can reference the same team for the same event and medal type, a GROUP BY on team code and url_event is applied, with MIN selecting a single medal_ID per group and MAX

selecting the is_medallist flag. The url_event column serves as a reliable grouping key since it uniquely identifies each event-level medal ceremony in the source data. This grouping strategy ensures that Team_wins contains one row per team-medal combination rather than one row per individual team member record in the medallists file.

```
INSERT INTO Team_wins (medal_ID, team_code, is_medallist)
SELECT MIN(m.medal_ID), ml.code_team, MAX(ml.is_medallist)
FROM MEDALLISTS ml
JOIN Event  e  ON e.event_name = ml.event AND e.event_type = ml.event_type
JOIN Awards aw ON aw.event_ID  = e.event_ID
JOIN Medal  m  ON m.medal_ID   = aw.medal_ID AND m.medal_type = ml.medal_type
WHERE ml.code_team IS NOT NULL
GROUP BY ml.code_team, ml.url_event;
```

Several implementation issues emerged during the population process. The first problem was structural in nature: the MEDALLISTS file simultaneously contains individual athlete records and team-based records, and in some situations both identifiers appear within the same row when an athlete is registered both individually and as part of a team delegation. Without explicitly separating the inserts through the conditions WHERE ml.code_athlete IS NOT NULL and WHERE ml.code_team IS NOT NULL, Oracle attempted to process all rows indiscriminately for both relationship tables. This produced foreign key violations because athlete identifiers were being inserted into team relationships and vice versa.

A second problem appeared specifically during the insertion into Team_wins. Since the original dataset stores one row for every athlete belonging to a medal-winning team, the insertion initially generated several duplicated rows representing the same team medal. Because the primary key of Team_wins only permits one record per (medal_ID, team_code) pair, Oracle raised repeated ORA-00001 unique constraint violations. The issue was resolved by grouping the rows using GROUP BY ml.code_team, ml.url_event. The inclusion of url_event was particularly important because it uniquely identifies each medal ceremony and therefore distinguishes medals awarded in different events even when the same team appears multiple times across the Games.

6.10 teams.csv

The teams.csv file models the organisational structure of Olympic team delegations and contributes information to five tables within the physical model: Team, Team_represents, Team_participates, Members, and Competes. Unlike simpler datasets that map directly to a single entity, this file encapsulates several distinct relationships simultaneously, requiring multiple insert operations and extensive data transformation during the import process.

The Team table was populated first using a SELECT DISTINCT projection over the fundamental identifying attributes of each team. Once the teams themselves were created, the Team_represents relationship was established directly through the country_code field already present in the CSV file, since the source data already uses valid NOC country abbreviations consistent with the Country table.

The Team_participates table required a join between the events column stored in TEAMS and the Event table. Because the file stores the textual event name rather than the surrogate event_ID, a direct equality join on the event name attribute was used to recover the corresponding event identifier.

The most complex population process involved the Members table, which represents the ternary relationship between athletes, coaches, and teams. The source dataset stores athlete and coach identifiers as concatenated strings within the columns athletes_codes and coaches_codes, rather than as separate rows. Consequently, membership could not be resolved through standard equality joins. Instead, Oracle's INSTR function was used to determine whether a particular athlete or coach code appeared within the corresponding string:

```
INSERT INTO Members (athlete_code, coach_code, team_code, num_athletes, num_coaches)
SELECT TO_NUMBER(a.code), TO_NUMBER(c.code),
    MIN(t.code),
    MAX(TO_NUMBER(t.num_athletes)),
    MAX(TO_NUMBER(t.num_coaches))
FROM TEAMS t
JOIN ATHLETES a ON INSTR(t.athletes_codes, a.code) > 0
JOIN COACHES  c ON INSTR(t.coaches_codes,  c.code) > 0
WHERE t.coaches_codes IS NOT NULL
GROUP BY a.code, c.code;
```

The import process revealed several important data quality and modelling problems. The first and most significant issue emerged in the Members insertion. The original implementation joined TEAMS, ATHLETES, and COACHES directly using the INSTR conditions but without aggregation. This produced multiple repeated athlete–coach combinations whenever the same pair appeared together in more than one team. Since the primary key of Members is defined as the combination of athlete_code and coach_code, Oracle raised repeated ORA-00001 unique constraint violations. The issue was resolved by grouping the results by athlete and coach identifiers and using aggregate functions such as MIN and MAX to collapse duplicate rows into a single relationship entry.

A final complication arose during the insertion into Competes. The source file identifies disciplines using a discipline code (disciplines_code), whereas the foreign key in the Competes table references the textual discipline name stored in the Discipline table. This mismatch required an intermediate join on the discipline code in order to retrieve the correct referenced value. Before performing the insertion, validation queries were executed to confirm that every discipline code present in TEAMS corresponded to an existing entry in the Discipline table, thereby preventing referential integrity violations.

6.11 training_sessions.csv

The training_sessions.csv file is the custom dataset created specifically for this project as required by the project specification. The file contains 72,232 rows representing athlete training sessions conducted during the three months immediately preceding the Olympic Games, covering the period from 11 May 2024 to 11 August 2024. The dataset was initially generated using Mockaroo and subsequently enriched and validated to ensure consistency with the remainder of the Olympic database schema.

The file contributes data to three tables in the physical model: Training, Trains, and Supervises. The Training entity table stores the core characteristics of each session, while the other two tables model the relationships between training sessions, athletes, and coaches.

The population of the Training table was performed directly from the staging file using a SELECT DISTINCT statement as a safeguard against duplicate identifiers generated during the synthetic data generation process:

```
INSERT INTO Training (training_ID, training_date, duration, intensity_level)
SELECT DISTINCT training_id, training_date, duration_min, intensity_level
FROM TRAINING_SESSIONS;
```

The Trains and Supervises relationship tables were considerably more straightforward to populate. Since the CSV file already contains the required athlete and coach identifiers associated with each training session, both tables could be populated through direct projections of the relevant columns without requiring additional joins:

```
INSERT INTO Trains (athlete_code, training_ID)
SELECT DISTINCT athlete_code, training_ID
FROM TRAINING_SESSIONS;

INSERT INTO Supervises (training_ID, coach_code)
SELECT DISTINCT training_ID, coach_code
FROM TRAINING_SESSIONS;
```

Although the CSV file also included a discipline column, this attribute was intentionally excluded from all insert operations. The discipline associated with a training session can already be derived indirectly through existing relationships between athletes, teams, and disciplines within the schema. Storing it again would therefore introduce unnecessary redundancy and violate the normalisation principles established during the database design phase.

7 Query implementation

7.1 Which athletes have won more medals than the average number of medals won per athlete across all disciplines?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, COUNT(w.medal_ID) AS
total_medals
FROM Athlete a
JOIN Person p ON a.athlete_code = p.person_code
JOIN Athlete_wins w ON a.athlete_code = w.athlete_code
WHERE w.is_medallist = 'True'
GROUP BY p.first_name, p.second_name, a.athlete_code
HAVING COUNT(w.medal_ID) > (
        SELECT AVG(medal_count)
        FROM (
                SELECT COUNT(w2.medal_ID) AS medal_count
                FROM Athlete_wins w2
                WHERE w2.is_medallist = 'True'
                GROUP BY w2.athlete_code
        )
)
ORDER BY total_medals DESC;
```

	ATHLETE_NAME	TOTAL_MEDALS
1	O'CALLAGHAN	5
2	Regan SMITH	5
3	Torri HUSKE	5
4	Yufei ZHANG	5
5	Junxuan YANG	4
6	Simone BILES	4
7	Gretchen WALSH	4
8	Rebeca ANDRADE	4
9	Kaylee McKEOWN	4
10	Katie LEDECKY	4
11	Shinnosuke OKA	4

7.1.1 Explanation

The query groups result by athlete because we need to aggregate all medals for each athlete individually. JOIN operations are necessary to properly connect athlete codes with the Person table and Athlete_wins table, that states if an athlete has won a medal. In both query and subquery, the variable is_medallist is checked to guarantee that the athlete in question has truly won a medal in any discipline. The subquery is used to calculate the average number of medals one by athletes throughout

all disciplines, which is used in comparison against the current athletes' medals in the HAVING clause. Overall, the average medals turned out to be 1.2, where a total of 194 athletes were returned with total medals varying from 2 to 5.

7.1.2 Validation

```
SELECT  p.first_name || ' ' || p.second_name AS athlete_name, w.MEDAL_ID
FROM Athlete a
JOIN Person p ON a.athlete_code = p.person_code
JOIN Athlete_wins w ON a.athlete_code = w.athlete_code
WHERE w.is_medallist = 'True' AND (UPPER(p.first_name) LIKE '%MOLLIE%');
```

	ATHLETE_NAME	MEDAL_ID
1	O'CALLAGHAN	43
2	O'CALLAGHAN M...	83
3	O'CALLAGHAN M...	11
4	O'CALLAGHAN M...	16
5	O'CALLAGHAN M...	9

To verify the validity of the query, an additional query was implemented to check if the total amount of medals per athlete was being calculated correctly. Specifically, we took one of the athletes from the table returned, in this case O'CALLAGHAN Mollie. As seen in the image above in the explanation section, this athlete returned a total amount of five medals one, where such result has been validated, as five different medal ids were returned for such athlete.

7.2 Which countries have managed to win medals in all three categories for individual athletes: gold, silver, and bronze?

```
SELECT c.country
FROM Country c
JOIN Person p ON c.country_code = p.nac_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins w ON a.athlete_code = w.athlete_code
JOIN Medal m ON w.medal_id = m.medal_id
WHERE w.is_medallist = 'True'
GROUP BY c.country
HAVING SUM(CASE WHEN m.medal_type = 'Gold Medal' THEN 1 ELSE 0 END) > 0 AND
    SUM(CASE WHEN m.medal_type = 'Silver Medal' THEN 1 ELSE 0 END) > 0 AND
    SUM(CASE WHEN m.medal_type = 'Bronze Medal' THEN 1 ELSE 0 END) > 0;
```

	COUNTRY
1	IR Iran
2	Georgia
3	Poland
4	Thailand
5	Israel
6	United States
7	Argentina
8	Croatia
9	Norway
10	Brazil
11	Azerbaijan

7.2.1 Explanation

This query is designed to find all countries that have at least won one of the three different medals that may be awarded, where their name is returned. Several JOIN operations are used to connect the Athlete with Country, as well as with the Medal table through Athlete_wins and know athletes that have won medals. Additionally, the WHERE clause allows to filter out all athletes that aren't medalists. The GROUP BY clause is necessary as the query performs aggregate calculations separately for each country. Lastly, cases are used to correctly aggregate the correct medal types and the HAVING verifies that such sum is greater than zero, meaning that such country does have at least one of those medal types.

7.2.2 Validation

```
SELECT c.country, SUM(CASE WHEN m.medal_type = 'Gold Medal' THEN 1 ELSE 0 END)
AS gold_count,
SUM(CASE WHEN m.medal_type = 'Silver Medal' THEN 1 ELSE 0 END) AS silver_count,
SUM(CASE WHEN m.medal_type = 'Bronze Medal' THEN 1 ELSE 0 END) AS bronze_count
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins w ON a.athlete_code = w.athlete_code
JOIN Medal m ON w.medal_id = m.medal_id
WHERE c.country IN ('IR Iran', 'Poland', 'United States') AND w.is_medallist = 'True'
GROUP BY c.country;
```

	COUNTRY	GOLD_COUNT	SILVER_COUNT	BRONZE_COUNT
1	United States	126	97	94
2	IR Iran	3	6	3
3	Poland	1	16	11

The result of the query was verified by taking three countries from those outputted in the queries result and their individual medals counts for each medal type were outputted. The image demonstrates countries who clearly pass the queries filtering such as United States or Poland which only has one golden medal which is still valid. The validation query is like the query 7.2, where instead of a HAVING clause, we directly select the aggregated amounts, filter by the countries we want to show and group by each country selected.

7.3 Which coaches are currently managing more athletes than the average number of athletes per coach?

```
SELECT p.first_name || ' ' || p.second_name AS coach_name, COUNT(m.athlete_code) AS
athlete_count
FROM Person p
JOIN Coach c ON p.person_code = c.coach_code
JOIN Members m on c.coach_code = m.coach_code
GROUP BY p.first_name, p.second_name, c.coach_code
HAVING COUNT(m.athlete_code) > (
                SELECT AVG(athletes_per_coach)
                FROM (
                        SELECT COUNT(m2.athlete_code) AS athletes_per_coach
                        FROM Members m2
                        GROUP BY m2.coach_code
                )
);
```

	COACH_NAME	ATHLETE_COUNT
1	Michael BRUUN	31
2	Eimear FLANNERY	28
3	Montse TOME	22
4	Arthur ELIAS	22
5	Rodrigo IGLESIAS	22
6	Futoshi IKEDA	22
7	Toshihiro NISHIIRI	22
8	David GORDO	22
9	Edson ALCANTARA	22
10	Santi DENIA	22
11	Tomomi MIYAMOTO	22
12	Javier LERGA	22

7.3.1 Explanation

Coaches that have a greater number of athletes under their command in comparison to the average number of athletes per coach will be returned for this query. To connect coaches with athletes, we connect them both through the Members' table, allowing them to retrieve athletes assigned to a single coach. GROUP BY allows us to count number of athletes specifically for each coach. The subquery allows us to identify the average number of athletes every coach manages throughout all disciplines, where each COUNT for all coaches is calculated and then averaged. Such average is then compared in the HAVING operation with the current coach being processed, allowing to only display coaches with number of athletes higher than the average.

7.3.2 Validation

```
SELECT COUNT(athlete_count) / COUNT (DISTINCT coach_code) AS average
FROM Members;
```

The average was also checked, which returned to a value of 14.4, demonstrating that coaches were filtered correctly, since the last coaches outputted in the query had a athlete count of 15.

```
SELECT p1.first_name || ' ' || p1.second_name AS coach_name, p2.first_name || ' ' ||
p2.second_name AS athlete_name
FROM Person p1
JOIN Coach c ON p1.person_code = c.coach_code
JOIN Members m ON c.coach_code = m.coach_code
JOIN Athlete a ON m.athlete_code = a.athlete_code
JOIN Person p2 ON a.athlete_code = p2.person_code
WHERE UPPER(p1.first_name) LIKE '%MICHAEL%' AND UPPER(p1.second_name) LIKE
'%BRUUN%';
```

	COACH_NAME	ATHLETE_NAME
1	Michael BRUUN	Emma FRIIS
2	Michael BRUUN	Mette HANSEN Anne
3	Michael BRUUN	Lukas JOERGENSEN
4	Michael BRUUN	Michala MOELLER
5	Michael BRUUN	Mette TRANBORG
6	Michael BRUUN	Henrik MOELLGAARD
7	Michael BRUUN	Lasse ANDERSSON
8	Michael BRUUN	Simon PYTLICK
9	Michael BRUUN	Louise VINTER BURGAARD

For this query, one of the coaches outputted were selected and all his athletes were returned. In this case, only a small number of Michael Bruun's athletes were shown, where he has a total of 31 different names.

7.4 Which 5 disciplines have a higher number of female participants than male participants?

```
SELECT d.discipline, SUM(CASE WHEN p.gender = 'Male' THEN 1 ELSE 0 END) AS
male_count, SUM(CASE WHEN p.gender = 'Female' THEN 1 ELSE 0 END) AS female_count
FROM Discipline d
JOIN Person_participates pp ON d.discipline_code = pp.discipline_code
JOIN Person p ON pp.person_code = p.person_code
GROUP BY d.discipline, d.discipline_code
HAVING female_count > male_count
ORDER BY female_count DESC
FETCH FIRST 5 ROWS ONLY;
```

	DISCIPLINE	MALE_COUNT	FEMALE_COUNT
1	Artistic Swimming	14	188
2	Rhythmic Gymna...	19	188
3	Canoe Sprint	119	120
4	Cycling Track	117	118
5	Table Tennis	87	88
6	3x3 Basketball	52	53

7.4.1 Explanation

This query compares male and female participation across different disciplines to identify such disciplines with a greater female presence, which may include athletes, coaches and technical officials. The JOIN operations allow us to connect the different Persons and their participation in different Disciplines. To properly organize data by discipline, the GROUP BY clause is used, allowing us to perform aggregation calculations on each category separately. Similarly, as in the query 7.2, cases are used to properly handle males and females, adding such values up and returning to different counts. The HAVING allows to filter out such disciplines that don't have more females than males, which is then organized by the greatest number of females to the least.

7.4.2 Validation

```
SELECT gender, COUNT(*)
FROM Person p
JOIN Person_participates pp ON p.person_code = pp.person_code
WHERE pp.discipline = 'Rhythmic Gymnastics'
GROUP BY gender;
```

	GENDER	COUNT(*)
1	Female	188
2	Male	19

Query 7.4 was validated by choosing a specific discipline and its individual counts for females and males was outputted. This demonstrates that one of the disciplines outputted from the query, has a larger count of females than males, since Rhythmic Gymnastics has 169 more females than males.

7.5 Which athletes have managed to win medals and participate in more than 10 unique events?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, COUNT(DISTINCT e.event_ID)
AS total_events
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Awards awd ON aw.medal_ID = awd.medal_ID
JOIN Athlete_participates pp ON a.athlete_code = pp.athlete_code
JOIN Event e ON pp.event_ID = e.event_ID
WHERE aw.is_medallist = 'True'
GROUP BY p.first_name, p.second_name, a.athlete_code
HAVING total_events > 10
ORDER BY total_events;
```

	ATHLETE_NAME	ATHLETE_CODE	TOTAL_EVENTS
1	Boheng ZHANG	1918864	13
2	Illia KOVTUN	1897555	13
3	Brody MALONE	1959823	13
4	Ruoteng XIAO	1918860	13
5	Jake JARMAN	1904741	13
6	Paul JUDA	1959855	13
7	Matthew RICHAR...	1940263	13
8	Shinnosuke OKA	1919480	13
9	Daiki HASHIMOTO	1919492	13
10	Frederick RICHARD	1959811	13
11	Summer McINTOSH	1967140	12
12	Takaaki SUGINO	1919479	12

7.5.1 Explanation

This query displays the name of the athlete and the number of distinct events he/she has participated in where they've won at least one medal in one of those events. The JOIN clauses allow us to connect the specific Person to the Athlete class, as well as connecting such code to the medal winners table, also ensuring that event information is accurately linked back to the athlete. Using a count of distinct event codes guarantees that we aren't counting several events multiple times but rather counting one per event linked to such athlete. Again, the WHERE clause allows to guarantee that the Athlete we're currently checking is a medalist with the variable is_medallist. Lastly, the HAVING clause makes sure that only athletes who have medaled and participated in more than 10 events are outputted.

7.5.2 Validation

```
SELECT e.event_ID
SELECT gender, COUNT(*)
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_participates pp ON a.athlete_code = pp.athlete_code
JOIN Event e ON pp.event_ID = e.event_ID
WHERE (p.first_name = 'Ruoteng' AND p.second_name = 'XIAO')
ORDER BY e.event_ID;
JOIN Person_participates pp ON p.person_code = pp.person_code
WHERE pp.discipline = 'Rhythmic Gymnastics'
GROUP BY gender;
```

	EVENT_ID
1	16
2	17
3	18
4	30
5	44
6	62
7	81
8	84
9	86
10	99
11	100
12	104
13	161

The query was validated by individually counting all the events that a specific athlete participated in. In this case, the athlete Ruoteng Xiao was tested, where a total of 13 distinct event_id's were returned, demonstrating that the count for her events is valid and wasn't modified when connecting other tables above to check if she was also a medalist.

7.6 Which venues have hosted events belonging to more than one different sport?

```
SELECT v.venue_name, COUNT(DISTINCT swc.sport_code) AS total_sports
FROM Venue v
JOIN Schedule sch ON v.venue_code = sch.venue
JOIN Event e ON sch.event_ID = e.event_ID
JOIN Showcases swc ON e.event_ID = swc.event_ID
JOIN Sport s ON swc.sport_code = s.sport_code
GROUP BY v.venue_name
HAVING total_sports > 1;
```

	VENUE_NAME	TOTAL_SPORTS
1	Yves-du-Manoir Stadium	12
2	South Paris Arena 1	12
3	Pierre Mauroy Stadium	12
4	South Paris Arena 6	13
5	Aquatics Centre	14
6	Stade de France	13
7	Bordeaux Stadium	12
8	Lyon Stadium	12
9	South Paris Arena 4	5
10	Bercy Arena	15

7.6.1 Explanation

In this specific query, several different JOIN clauses are required to connect venues to scheduled events, events to their specific sport and sport to their identifying information, allowing to guarantee that the data fits correctly. The use of the COUNT for distinct sport codes ensures that each sport is only counted once per venue, even if multiple events for the same sport occur in the same location. Grouping by the venue_name guarantees that we're individually checking if each venue has hosted more than one sport or not. Due to total_sports being an aggregated function in the first SELECT, a HAVING operator is used to confirm if the venue in question has hosted more than one sport. Finally, a total of 32 venues are returned that host multiple different sports.

7.6.2 Validation

```
SELECT v.venue_name, s.sport, COUNT(DISTINCT sch.event_ID) AS events
FROM Venue v
JOIN Schedule sch  ON v.venue_code = sch.venue
JOIN Showcases swc ON sch.event_ID = swc.event_ID
JOIN Sport s ON swc.sport_code = s.sport_code
WHERE v.venue_name <= ALL (
        SELECT v2.venue_name
        FROM Venue v2
        JOIN Schedule sch2 ON v2.venue_code = sch2.venue
        JOIN Showcases swc2 ON sch2.event_ID = swc2.event_ID
        GROUP BY v2.venue_name
        HAVING COUNT(DISTINCT swc2.sport_code) > 1
        )

GROUP BY v.venue_name, s.sport;
```

	VENUE_NAME	SPORT	EVENTS
1	Aquatics Centre	Cycling BMX Raci...	2
2	Aquatics Centre	3x3 Basketball	2
3	Aquatics Centre	Beach Volleyball	2
4	Aquatics Centre	Hockey	2
5	Aquatics Centre	Water Polo	2
6	Aquatics Centre	Surfing	2
7	Aquatics Centre	Football	2
8	Aquatics Centre	Rugby Sevens	2
9	Aquatics Centre	Trampoline Gymn...	2
10	Aquatics Centre	Volleyball	2
11	Aquatics Centre	Basketball	2
12	Aquatics Centre	Handball	2
13	Aquatics Centre	Diving	8
14	Aquatics Centre	Artistic Swimming	2

To verify the validity of the query, one of the venues' outputted was chosen, in this case the Aquatics center. A total of 14 rows with distinct sports is returned, which coincides with the total amount returned in the original query. Additionally, this result is further justified by the number of events completed for such sports in this specific venue.

7.7 Which athletes representing Spain have also studied in University at Spain?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, e.education
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Attended att ON a.athlete_code = att.athlete_code
JOIN Education e ON att.education_ID = e.education_ID
WHERE (UPPER(e.education) LIKE '%UNIVERSITY%' AND UPPER(e.education) LIKE
'%(ESP)%') AND p.noc_code = 'ESP';
```

	ATHLETE_NAME	EDUCATION
1	Cecilia CASTRO BURGOS	Faculty of Psychology at the Autonomous University of Madrid (ESP)
2	Adrian VICENTE YUNTA	Studied Physical Education at Polytechnic University of Madrid (ESP)
3	Paula SORIA GUTIERREZ	Graduated with a Bachelor's degree in Criminology from the University of Alicante (E...
4	Jaime CANALEJO PAZOS	Graduated with a Bachelor's degree in Sports Science from University of Seville (ESP)
5	MARTIN-PORTUGUES L...	Studies Dentistry at Complutense University of Madrid (ESP)
6	Jael BESTUE	Medicine at University of Barcelona (ESP)
7	Marta GARCIA	Medicine at University of Valladolid (ESP)
8	Raquel GONZALEZ	Studying for a MBA at UCAM, Catholic University of Murcia (ESP). Graduate...
9	Lorea IBARZABAL	Mechanical Engineering/Mathematics at University of Portland (Portland, OR, USA). P...
10	Angel LOPEZ Miguel	Sports Science - Saint Anthony Catholic University, Murcia (ESP)
11	Alvaro MARTIN	Political Science at Complutense University of Madrid (ESP). Also studies for a law de...
12	Paul McGRATH	Journalism at University of Barcelona (ESP)
13	Maria PEREZ	Teaching at Catholic University of Murcia (ESP)
14	Marta PEREZ	Medicine at Complutense University of Madrid (ESP)
15	Yago ROJO	Mechanical Engineering at Madrid Polytechnic University (ESP)

7.7.1 Explanation

The query joins people, athletes and education to determine which athletes representing Spain have also gone or are attending university in Spain as well. Each JOIN allows us to trace the necessary information to connect the athlete's education to the athlete himself, as well as using the foreign key of the person's noc code. The WHERE clause applies to the key condition that compares the athlete's education to the words UNIVERSITY and (ESP), allowing to filter out athletes who haven't gone to university or haven't studied in Spain. Grouping in this case isn't required since no aggregation is performed, where the query simply returns row matches. A total of 54 athletes is returned to fulfil the three requirements mentioned in the title queries.

7.7.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, p.noc_code
        FROM Person p
        JOIN Athlete a ON p.person_code = a.athlete_code
        WHERE (UPPER(p.first_name) = 'SAUL' AND UPPER(p.second_name) =
        'CRAVIOTTO') OR (UPPER(p.first_name) = 'ALVARO' AND UPPER(p.second_name)
        = 'MARTIN'):
```

	ATHLETE_NAME	NOC_CODE
1	Alvaro MARTIN	ESP
2	Saul CRAVIOTTO	ESP

In this case the validation consisted in checking if truly the athletes that were being returned, their noc codes were the same as the one found in the education attribute. By taking two different athletes, Saul Craviotto and Alvaro Martin, it's demonstrated they do represent Spain in the Olympic games.

7.8 Which coaches are training athletes across more than one discipline at the same time?

```
SELECT p.first_name || ' ' || p.second_name AS coach_name, c.coach_code,
COUNT(DISTINCT pp.discipline) AS training_disciplines
FROM Person p
JOIN Coach c ON p.person_code = c.coach_code
JOIN Members m ON c.coach_code = m.coach_code
JOIN Athlete a ON m.athlete_code = a.athlete_code
JOIN Person_participates pp ON a.athlete_code = pp.person_code
GROUP BY p.first_name, p.second_name, c.coach_code
HAVING COUNT(DISTINCT pp.discipline) > 1;
```

	COACH_NAME	COACH_CODE	TRAINING_DISCIP
1	Sandy BRONDE	1940093	2
2	Cheryl CHAMBERS	1940075	2

7.8.1 Explanation

The query groups result by coach because we want to determine the number of coaches that are training athletes across different disciplines. Multiple JOIN clauses are called to connect coaches to their team members, athletes to their participation in events log, and training sessions to ensure all discipline information is correctly linked to the coach. The use of *COUNT*(DISTINCT ap.discipline_code) guarantees that each discipline is only counted once per coach, allowing you to ignore the possibility that a coach might have many athletes from the same discipline. Since training_disciplines is an aggregated result, the HAVING allows you to finally verify and filter such coaches that don't train athletes in more than one discipline.

7.8.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS coach_name, pp.discipline
        FROM Person p
        JOIN Coach c ON p.person_code = c.coach_code
        JOIN Members m ON c.coach_code = m.coach_code
        JOIN Athlete a ON m.athlete_code = a.athlete_code
        JOIN Person_participates pp ON a.athlete_code = pp.person_code
        WHERE c.coach_code = 1940093;
```

	COACH_NAME	DISCIPLINE
1	Sandy BRONDE	Basketball
2	Sandy BRONDEL...	3x3 Basketball

To validate the discipline_count for each coach, one of the output coaches was chosen to print all the disciplines linked to its coach_code. For Sandy Brondello, a total of two different disciplines were returned, which may coincide with reality since both disciplines are related but different.

7.9 Which countries have sent more athletes to compete than the average number of athletes per country?

```
SELECT c.country, COUNT(a.athlete_code) AS athlete_count
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
GROUP BY c.country, c.noc_code
HAVING COUNT(a.athlete_code) > (
        SELECT AVG(athletes_per_country)
        FROM(
                SELECT COUNT(athlete_code) AS athletes_per_country
                FROM Person p2
                JOIN Athlete a2 ON p2.person_code = a2.athlete_code
                GROUP BY p2.noc_code
          )
)
ORDER BY athlete_count DESC;
```

	COUNTRY	ATHLETE_COUNT
1	United States	619
2	France	601
3	Australia	475
4	Germany	457
5	Japan	431

	COUNTRY	ATHLETE_COUNT
45	Cuba	61
46	Morocco	61
47	Chinese Taipei	60
48	Dominican Repub...	59
49	Finland	57

7.9.1 Explanation

The following query groups results by country, since we want to calculate how many athletes each country has contributed to the Olympic games. The JOIN operations connect countries to their corresponding athletes, ensuring that each athlete is correctly associated with its national committee. A subquery is used to calculate the average number of athletes sent by countries, which serves as a benchmark for comparison. Since athlete_count is an aggregated result, the HAVING operation allowed filtering countries only after grouping, outputting the countries with higher athlete numbers than the average.

7.9.2 Validation

```
SELECT COUNT(a.athlete_code) / COUNT(DISTINCT p.noc_code) AS global_avg
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code;
```

	GLOBAL_AVG
1	53.94660194174...

On one hand, the global average of athletes per country was calculated, which returned to an average of 53.94, proving that all 49 countries output in the query have a total count of athletes greater than the global average.

```
SELECT c.country, p.first_name || ' ' || p.second_name AS athlete_name
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
WHERE c.country = 'Finland'
GROUP BY c.country, p.first_name, p.second_name;
```

	COUNTRY	ATHLETE_NAME
1	Finland	Antti TEKONIEMI
2	Finland	Kalle KOLJONEN
3	Finland	KAIVO-OJA Pihla
4	Finland	Anniina AHTOSALO
5	Finland	Noora KOMULAIN...
6	Finland	Tapio PULKKANEN
7	Finland	Sami VALIMAKI
8	Finland	Matti MATTSSON
9	Finland	Kunnari SARKKINE...
10	Finland	Martin SAVOLAIN...
11	Finland	Ella JUNNILA
12	Finland	Lotta KEMPPINEN

On the other hand, for one of the countries' outputted, we validated that the number of athletes returned is the true value of athletes. For the country Finland, a total of 57 distinct athlete names was returned, truly demonstrating that the count returned in the query was also valid.

7.10 Which sports have had their events spread across the higher number of different venues?

```
SELECT sp.sport, COUNT(DISTINCT sch.venue) AS venue_count
FROM Sport sp
JOIN Showcases swc ON sp.sport_code = swc.sport_code
JOIN Schedule sch ON swc.event_ID = sch.event_ID
GROUP BY sp.sport, sp.sport_code
ORDER BY venue_count DESC
FETCH FIRST 5 ROWS ONLY;
```

	SPORT	VENUE_COUNT
1	3x3 Basketball	19
2	Cycling BMX Raci...	19
3	Water Polo	19
4	Volleyball	19
5	Trampoline Gymn...	19

7.10.1 Explanation

COUNT(DISTINCT s.venue_name) was used to ensure that each venue is counted only once per sport, which doesn't restrict the fact that possible such sport has had multiple events in such venue. The query groups result by sport because the goal is to find howe many different venues each sport has. In this case, JOIN operators are used to connect the events of a specific sport to different entries of the schedule. To identify which sports have had the most different venues in their scheduled events, venue_count is ordered in descending order.

7.10.2 Validation

```
SELECT sp.sport, v.venue_name
FROM Sport sp
JOIN Showcases swc ON sp.sport_code = swc.sport_code
JOIN Schedule sch ON swc.event_ID = sch.event_ID
JOIN Venue v ON sch.venue = v.venue_code
WHERE sp.sport = '3x3 Basketball'
GROUP BY sp.sport, v.venue_name;
```

	SPORT	VENUE_NAME
12	3x3 Basketball	Parc des Princes
13	3x3 Basketball	Nice Stadium
14	3x3 Basketball	La Beaujoire Stadium
15	3x3 Basketball	Marseille Stadium
16	3x3 Basketball	Teahupo'o, Tahiti
17	3x3 Basketball	La Concorde 1
18	3x3 Basketball	BMX Stadium
19	3x3 Basketball	Paris La Defense Arena

The following query was provided to verify the validity of query 7.10. For the sport 3x3 Basketball, all venue names related to a scheduled event for 3x3 basketball were returned. The total number of different venue names came out to a total of 19 rows, the same count as output in the query above, proving the counts are valid.

7.11 Which athletes share the same personal hero (father) and have also won at least one medal?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, per.hero
FROM Athlete a
JOIN Person p ON a.athlete_code = p.person_code
JOIN Personality per ON a.personality_ID = per.personality_ID
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE (LOWER(per.hero) LIKE '%father%' OR LOWER(per.hero) LIKE '%dad%') AND
aw.is_medallist = 'True' AND per.hero IS NOT NULL;
```

	ATHLETE_NAME	HERO
1	Sarabjot SINGH	His father
2	Michael JUNG	His father Joachi...
3	Gabriele ROSSETTI	His father, shoot...
4	Bartosz KUREK	His father, volley...
5	Sagen MADDALE...	Her father
6	Jan van der BIJ	His father
7	Pierre BROL CAR...	His father

7.11.1 Explanation

In this query, athletes, their personal information, their personality and medal records are joined to identify individuals that have won medals that share the same hero figure: their father. The WHERE clause in this query filters results by firstly checking for keywords such as “father” or “dad” in the hero field, using LOWER() and LIKE to make the search case-sensitive and flexible. Additionally, we make sure that only true medalists will be output, by checking if is_medallist is true. Lastly, it must be also considered and checked that the hero field isn’t empty, which occurs in many occasions.

7.11.2 Validation

```
SELECT COUNT(DISTINCT a.athlete_code) AS medalist_with_father_hero
FROM Athlete a
JOIN Personality per ON a.personality_ID = per.personality_ID
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE (LOWER(per.hero) LIKE '%father%' OR LOWER(per.hero) LIKE '%dad%') AND
aw.is_medallist = 'True' AND per.hero IS NOT NULL;
```

	MEDALIST_WITH_
1	7

The first validation query produced was validating if truly only seven athletes whose hero is their data has also won a medal in the games. The count produced for distinct athletes, connecting their personalities and winning a medal table, shows that truly only seven athletes apply for such case.

```
SELECT per.hero
FROM Athlete a
JOIN Person p ON a.athlete_code = p.person_code
JOIN Personality per ON a.personality_ID = per.personality_ID
WHERE p.second_name LIKE '%JUNG%';
```

	HERO
1	His father Joachim Jung

The second validation query computed was truly verifying that the athlete's output, their hero is truly their father or if the query didn't filter out correctly the hero attribute. By taking the name of one of the output athletes in the query, their hero attribute as itself was shown, demonstrating that their hero in this case is truly their father, since additionally they also share the same last name.

7.12 Which disciplines include both athletes born in the same country they represent and athletes born in a different country from the one they represent?

```
SELECT d.discipline,
COUNT(DISTINCT CASE WHEN p.birth_country = c.noc_code THEN a.athlete_code END)
AS native_athletes,
COUNT(DISTINCT CASE WHEN p.birth_country <> c.noc_code THEN a.athlete_code END)
AS foreign_athletes
FROM Discipline d
JOIN Person_participates pp ON d.discipline = pp.discipline
JOIN Person p ON pp.person_code = p.person_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Country c ON p.noc_code = c.noc_code
WHERE p.birth_country IS NOT NULL
GROUP BY d.discipline
HAVING native_athletes > 0 AND foreign_athletes > 0
ORDER BY native_athletes DESC;
```

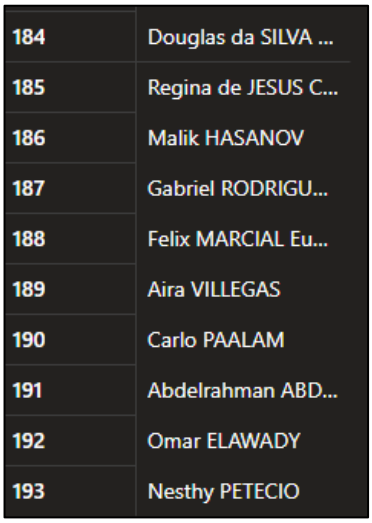
	DISCIPLINE	NATIVE_ATHLETES	FOREIGN_ATHLETE
1	Athletics	1314	144
2	Swimming	473	47
3	Football	428	53
4	Handball	337	32
5	Rowing	309	17
6	Hockey	295	15
7	Judo	288	67
8	Sailing	272	26
9	Shooting	250	29
10	Volleyball	225	12
11	Boxing	193	22
12	Rugby Sevens	189	15
13	Basketball	181	16
14	Water Polo	180	24
15	Cycling Track	175	7

7.12.1 Explanation

This query groups results by discipline, since the goal is to find all disciplines who have athletes who were born in the country, they compete for and those athletes representing a country which they weren't born in. JOIN conditions allow us to connect discipline with all the athletes that compete in it and consecutively these athletes with their country's information. On one hand, we have the count for native athletes, since we want to count all the athletes who represent their birth country, we have *SUM*(DISTINCT CASE WHEN p.birth_country = c.noc_code THEN a.athlete_code END), where using distinct case allows to take all athletes once and only count them if their country noc code is equal to their birth country. On the other hand, we have an exact same count for athletes who represent a country that isn't the one they were born in, where in this case CASE allows them to add to the count if noc and birth country aren't equal. The HAVING clause allows to guarantee that the two aggregated are greater than 0, allowing them to return only disciplines that include both types of athletes.

7.12.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Person_participates pp ON a.athlete_code = pp.person_code
WHERE pp.discipline = 'Boxing' AND p.birth_country = p.noc_code;
```



184	Douglas da SILVA ...
185	Regina de JESUS C...
186	Malik HASANOV
187	Gabriel RODRIGU...
188	Felix MARCIAL Eu...
189	Aira VILLEGAS
190	Carlo PAALAM
191	Abdelrahman ABD...
192	Omar ELAWADY
193	Nesthy PETECIO

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Person_participates pp ON a.athlete_code = pp.person_code
WHERE pp.discipline = 'Boxing' AND p.birth_country <> p.noc_code;
```

14	Charlie SENIOR
15	Yusuf CHOTHIA
16	Pyly AKILOV
17	Michaela WALSH
18	Natalia SHADRINA
19	Vakhid ABBASOV
20	Berto ALFONSO D...
21	Jude GALLAGHER
22	PLODZICKI- FAOA...

To prove that athlete counts made for each discipline output were valid, two validation queries were formulated for the two different counts. Tables of Person, Athlete, and Discipline were joined to individually display only the athletes that we want for the specific discipline. For the discipline of boxing, all athletes who represent the country they were born in were first formulated, outputting all the rows of distinct athletes, which came out to a total of 193 different rows, the same count as above. Additionally, the number of rows for athletes not representing their birth country came out to 22 rows, also being the same count as for the query.

7.13 Which athletes were born in the same country they're representing and have won at least one gold medal?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, c.country, p.birth_country,
COUNT(aw.medal_ID) AS gold_medal
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Country c ON p.noc_code = c.noc_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Medal m ON aw.medal_ID = m.medal_ID
WHERE p.birth_country = c.noc_code AND m.medal_type = 'Gold Medal'
GROUP BY p.person_code, p.first_name, p.second_name, c.country, p.birth_country
ORDER BY gold_medal DESC;
```

	ATHLETE_NAME	COUNTRY	BIRTH_COUNTRY	GOLD_MEDAL
1	Shinnosuke OKA	Japan	JPN	3
2	Harrie LAVREYSEN	Netherlands	NED	3
3	Leon MARCHAND	France	FRA	3
4	O'CALLAGHAN M...	Australia	AUS	3
5	Summer McINTOSH	Canada	CAN	3
6	Lisa CARRINGTON	New Zealand	NZL	3
7	Alicia HOSKIN	New Zealand	NZL	2
8	Sarah SJOESTROEM	Sweden	SWE	2
9	Edriel YULO Carlos	Philippines	PHI	2
10	BREDOW-WERND...	Germany	GER	2
11	Shayna JACK	Australia	AUS	2
12	Beatrice CHEBET	Kenya	KEN	2
13	Max LEMKE	Germany	GER	2

7.13.1 Explanation

To properly return to the athletes who have won a medal for the country they were born in, athletes, countries and medal record tables were joined. The JOIN clauses make sure that the athlete is correctly linked to their personal information and medal history. The WHERE clause must check two conditions: that the birth country is equal to the country of the associated NOC code of the athlete, and that the medal associated with the athlete is gold. Lastly, due to the aggregate function of counting gold medals for the athlete, we must GROUP BY for the athlete itself as well as the country.

7.13.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, m.medal_type, m.medal_ID
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Medal m ON aw.medal_ID = m.medal_ID
WHERE p.first_name = 'Shinnosuke'
AND p.second_name = 'OKA'
AND m.medal_type = 'Gold Medal';
```

	ATHLETE_NAME	MEDAL_TYPE	MEDAL_ID
1	Shinnosuke OKA	Gold Medal	4
2	Shinnosuke OKA	Gold Medal	63
3	Shinnosuke OKA	Gold Medal	119

The validation in this case was performed by creating a query that prints out all the golden medals of a specific athlete that was returned. In this case, Shinnosuke Oka returned a total count of three different golden medals, where the validation query proves that she indeed win more than one golden medal and they all have a different id.

7.14 Which disciplines have an average athlete age that is higher than the global average age across all disciplines?

```
SELECT pp.discipline, ROUND(AVG(MONTHS_BETWEEN(SYSDATE, p.birth_date) / 12),
1) AS avg_age
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Person_participates pp ON p.person_code = pp.person_code
WHERE p.birth_date IS NOT NULL
GROUP BY pp.discipline
HAVING AVG(MONTHS_BETWEEN(SYSDATE, p.birth_date) / 12) > (
                SELECT AVG(MONTHS_BETWEEN(SYSDATE, p2.birth_date) / 12)
                FROM Person p2
                JOIN Athlete a2 ON p2.person_code = a2.athlete_code
                WHERE p2.birth_date IS NOT NULL
)
ORDER BY avg_age DESC;
```

	DISCIPLINE	AVG_AGE
1	Equestrian	41.2
2	3x3 Basketball	31.9
3	Beach Volleyball	31.7
4	Shooting	31.5
5	Golf	31.4
6	Tennis	31.1
7	Triathlon	30.7
8	Handball	30.7
9	Table Tennis	30.6
10	Volleyball	30.3
11	Canoe Slalom	30.3

7.14.1 Explanation

In this query multiple aggregation functions are used to compare the average athlete ages for each discipline to the global average age of athletes. The joins ensure that the athlete is correctly linked to his personal information as well as the discipline they participate in. The subquery inside the WHERE clause computes the discipline specific average age, which is compared to the overall athlete age. Because the comparison is made at the discipline level, it correctly filters out all disciplines that have an average lower to the global average.

7.14.2 Validation

```
SELECT ROUND(AVG(MONTHS_BETWEEN(SYSDATE, p.birth_date) / 12), 1) AS
global_avg_age
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
WHERE p.birth_date IS NOT NULL;
```

	GLOBAL_AVG_AGE
1	28.8

To verify that the query filtered correctly those disciplines with an average age higher than the global average, a small query was produced to find such an average. All 23 rows output for the query had an average age higher than 28.8, meaning the filtering when comparing averages was effectively done.

```
SELECT pp.discipline, ROUND(AVG(MONTHS_BETWEEN(SYSDATE, p.birth_date) / 12),
1) AS avg_age
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Person_participates pp ON p.person_code = pp.person_code
WHERE pp.discipline = 'Equestrian' AND p.birth_date IS NOT NULL
GROUP BY pp.discipline;
```

	DISCIPLINE	AVG_AGE
1	Equestrian	41.2

A secondary query was written to verify that the average age was correctly computed for each different discipline. In this case, Equestrian's average age for athletes was 41.2, the same value as in for the original query, proving that calculations are being computed correctly.

7.15 Which organizations have technical officials involucrate in more than ten different disciplines?

```
SELECT tof.organisation, COUNT(DISTINCT d.discipline) AS disciplines_covered
FROM Technical_Official tof
JOIN Person p ON tof.tech_code = p.person_code
JOIN Person_participates pc ON p.person_code = pc.person_code
JOIN Discipline d ON pc.discipline = d.discipline
GROUP BY tof.organisation
HAVING COUNT(DISTINCT d.discipline) > 10
ORDER BY disciplines_covered DESC;
```

	ORGANISATION	DISCIPLINES_COVE
1	FRA	18
2	JPN	15
3	GER	13
4	USA	13
5	AUS	12
6	ITA	11
7	ESP	11

7.15.1 Explanation

This query groups results by organization, since we want to determine how many distinct disciplines each organization's officials participate in. *COUNT*(DISTINCT d.discipline) ensures that we correctly determine the discipline associated with each organization. The joins connect each organization to its officials that participate in different disciplines, where only one official per discipline won will be considered at the time of the query. Results were ordered in descending order

of disciplines covered to first output the organizations with the most unique discipline coverage. Finally, HAVING allows us to filter out all those organizations who haven't provided technical officials across more than 10 unique disciplines.

7.15.2 Validation

```
SELECT DISTINCT d.discipline
FROM Technical_Official tof
JOIN Person p ON tof.tech_code = p.person_code
JOIN Person_participates pc ON p.person_code = pc.person_code
JOIN Discipline d ON pc.discipline = d.discipline
WHERE tof.organisation = 'FRA'
ORDER BY d.discipline;
```

	DISCIPLINE
1	Artistic Gymnastics
2	Beach Volleyball
3	Boxing
4	Breaking
5	Cycling BMX Freestyle
6	Diving
7	Football
8	Handball
9	Judo
10	Rhythmic Gymnastics
11	Rugby Sevens
12	Skateboarding
13	Surfing
14	Trampoline Gymnastics
15	Volleyball
16	Water Polo
17	Wrestling
18	3x3 Basketball

The query was validated by validating that the count for distinct disciplines for the organizations is valid. In this case, the French organization was taken and all its distinct disciplines it covers by its officials was outputted, demonstrating 18 total rows, the same count as for the original query.

7.16 Which athletes have won at least one medal and also have a sister who also does sports?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Is_Family_Of ifo ON p.person_code = ifo.family_code
WHERE aw.is_medallist = 'True' AND ifo.is_sporting_relative = 'False' AND
LOWER(ifo.family_member) LIKE '%sister%';
```

	ATHLETE_NAME
1	Sebastian HEYN
2	Ogechi EGONU P...
3	Yuting HUANG
4	Ariarne TITMUS
5	Diyora KELDIYOR...
6	Sunisa LEE
7	Inbar LANIR
8	Lynn PRINCE Con...
9	Kristen FAULKNER
10	Alison dos SANT...
11	Viviana MARTON

7.16.1 Explanation

This query filters the dataset to find those athletes who have won medals and identify those athletes who have a sporting relative described as their sister. The JOINs allow the Athlete to link with his personal information, medal records as well connecting it with the reflexive table Is_Family_Of. Requiring is_sporting_relative = 'True' guarantees that the family member in question is also into sports and isn't just a family member. The *LOWER*(ifo.family_member) LIKE '%sister%' condition allows in a case-insensitive way to check for the key word sister in the TEXT type attribute of family_member. Total of 230 rows were returned of athletes who have a sister as a sporting relative, as well as them being medalists.

7.16.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, ifo.family_member AS
family_info, ifo.is_sporting_relative
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Is_Family_Of ifo ON p.person_code = ifo.family_code
WHERE p.second_name LIKE '%MATIC%';
```

	ATHLETE_NAME	FAMILY_INFO	IS_SPORTING_REI
1	Barbara MATIC	Sister, Brigita	True

In this case, the query was validated by checking if the output athlete truly has a sister, as well as if such sister is an athlete herself. In this case multiple Joins were performed to join the athlete to his family info and if such family member is a sporting relative. Barbara Matic proved to have an athlete sister, as she was one of the outputted athletes in the query.

7.17 Which countries have athletes competing in every discipline currently scheduled in the Olympic Games?

```
SELECT c.country , COUNT(DISTINCT pp.discipline) AS disciplines_total
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Person_participates pp on p.person_code = pp.person_code
JOIN Country c ON p.noc_code = c.noc_code
GROUP BY c.country
HAVING disciplines_total = (SELECT COUNT(*) FROM Discipline);
```

COUNTRY	DISCIPLINES_TOT
No se ha encontrado ningún dato	

7.17.1 Explanation

In this scenario, the query will return the name of the country where for all the disciplines currently in the Olympic games, it has athletes competing, where results will be grouped by country. Each athlete is correctly linked to their nationality and participation records to the specific discipline. The HAVING clause is used to compare the total count of disciplines that the country participates in the total number of disciplines of the dataset. For this query, it isn't necessary to order the information outputted by the disciplines_total, since such value if correct will be the same number for all countries. The query returned to a total of zero countries, meaning that no countries have an athlete in every single discipline.

7.17.2 Validation

```
SELECT COUNT(*) AS total_disciplines
FROM Discipline;
```

	TOTAL_DISCIPLINES
1	47

Firstly, to validate the result, the total number of disciplines was checked, which came out to a total amount of 47. To check if all other countries had less than such value, the same query as above was executed but without the HAVING clause.

```
SELECT c.country , COUNT(DISTINCT pp.discipline) AS disciplines_total
FROM Person p
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Person_participates pp on p.person_code = pp.person_code
JOIN Country c ON p.noc_code = c.noc_code
GROUP BY c.country;
```

	COUNTRY	DISCIPLINES_TOTAL
1	France	45
2	United States	44
3	Australia	42
4	China	42
5	Japan	41
6	Canada	40
7	Germany	40
8	Spain	38
9	Italy	37
10	Brazil	37

This validation query demonstrates that none of the countries in the Olympic games had athletes participating in all disciplines, where France came close to 45 total disciplines with French athletes. Therefore the filtering performed in the original query, correctly eliminated all countries, since none were valid responses.

7.18 Which coaches have never had any of their athletes win a medal throughout the competition?

```
SELECT p.first_name || ' ' || p.second_name AS coach_name
        FROM Person p
JOIN Coach c ON p.person_code = c.coach_code

MINUS

SELECT p.first_name || ' ' || p.second_name AS coach_name
FROM Person p
JOIN Coach c ON p.person_code = c.coach_code
JOIN Members m ON c.coach_code = m.coach_code
JOIN Athlete a ON m.athlete_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE aw.is_medallist = 'True';
```

7.18.1 Explanation

The query first retrieves the full list of coaches in the dataset by joining the Person and Coach tables. The second query identifies coaches who have worked with athletes who have won medals, by joining the Members, Athlete, and Athlete_wins tables to trace each coach's athlete's and their medal results. The MINUS operator subtracts this second query from the first, leaving only coaches who have no medal-winning athletes associated with them. This approach ensures that even if a coach has many athletes, even if one of them won a medal, such coach is immediately removed.

7.18.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS coach_name,
COUNT(DISTINCT CASE WHEN aw.is_medallist = 'True' THEN a.athlete_code END
) AS medal_athletes
FROM Person p
JOIN Coach c ON p.person_code = c.coach_code
LEFT JOIN Members m ON c.coach_code = m.coach_code
LEFT JOIN Athlete a ON m.athlete_code = a.athlete_code
LEFT JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE p.first_name = 'Olesia'
  AND p.second_name = 'ZAITSEVA'
GROUP BY p.first_name, p.second_name;
```

	COACH_NAME	MEDAL_ATHLETES
1	Olesia ZAITSEVA	0

This case is taking part of the query from the original but not taking the query above the minus that takes all coaches. Specifically in this case, the count must have a CASE inside it since if not it wouldn't return an actual valid output, since other athletes were also considered. Additionally, for the query to get those athletes who haven't won a medal, LEFT JOINS were utilized, since these athletes have nulls for the athlete_wins table.

7.19 Which nationalities appear most frequently among the athletes who have won gold medals?

```
SELECT c.country, COUNT(aw.medal_ID) AS gold_medals
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Medal m ON aw.medal_ID = m.medal_ID
WHERE aw.is_medallist = 'True' AND m.medal_type = 'Gold Medal'
GROUP BY c.country
ORDER BY gold_medals DESC
FETCH FIRST 10 ROWS ONLY;
```

	COUNTRY	GOLD_MEDALS
1	United States	126
2	China	70
3	Netherlands	65
4	France	51
5	Spain	40
6	Great Britain	37
7	Australia	32
8	Italy	31
9	Japan	27
10	New Zealand	26

7.19.1 Explanation

The query filters medal records to include only gold medals, as well as linking each medal to the athlete and their country consecutively. The JOIN with the table medal ensures that we're capable of isolating golden medals through the metal_type attribute. Grouping by country aggregates the total number of golden medals for all athletes for a respective country, which such athletes must be medalists, checked through is_medallist. Results are ordered by gold_medals, in order to highlight the countries with the strongest gold medal performance in the Olympic games.

7.19.2 Validation

```
SELECT c.country, p.first_name || ' ' || p.second_name AS athlete_name
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Medal m ON aw.medal_ID = m.medal_ID
WHERE aw.is_medallist = 'True' AND m.medal_type = 'Gold Medal' AND c.country = ‘Italy'
GROUP BY athlete_name;
```

	COUNTRY	ATHLETE_NAME
1	Italy	Alberta SANTUCCIO
2	Italy	Alessia ORRO
3	Italy	Alice BELLANDI
4	Italy	Anna DANESI
5	Italy	Carlotta CAMBI
6	Italy	Chiara BOSETTI Ca...
7	Italy	Chiara CONSONNI
8	Italy	D'AMATO Alice
9	Italy	Diana BACOSI
10	Italy	Ekaterina ANTROP...
11	Italy	Fatime SYLLA Myri...
12	Italy	Gabriele ROSSETTI
13	Italy	Gaia GIOVANNINI

A validation query for one of the countries outputted was constructed to show all the athletes who have won golden medals, in this case Italy. In this scenario, 31 rows were outputted, meaning an exact amount of 31 athletes each won one golden medal for Italy, which perfectly fits the result returned in the query for question 19.

7.20 Which disciplines have the lowest total number of athletes registered to compete?

```
SELECT pp.discipline, COUNT(DISTINCT pp.person_code) AS total_athletes
FROM Person_participates pp
JOIN Athlete a ON pp.person_code = a.athlete_code
GROUP BY pp.discipline
ORDER BY total_athletes ASC
FETCH FIRST 10 ROWS ONLY;
```

	DISCIPLINE	TOTAL_ATHLETES
1	Cycling BMX Fre...	24
2	Trampoline Gymn...	32
3	Breaking	33
4	Surfing	48
5	Marathon Swimm...	55
6	3x3 Basketball	65
7	Cycling BMX Raci...	67
8	Sport Climbing	68
9	Cycling Mountain...	70
10	Modern Pentathlon	72

7.20.1 Explanation

The query links each discipline to the Person registered under it through the Person_participates table, where then is linked to the Athlete table. Grouping by discipline allows you to correctly check how many distinct athletes are associated with each discipline. Using the COUNT for distinct person_code ensures that each athlete is counted exactly once even if they appear multiple times. Sorting by the total_athletes in ascending order guarantees that the discipline with the least athletes is at the top of the column when output.

7.20.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name
FROM Person_participates pp
JOIN Athlete a ON pp.person_code = a.athlete_code
JOIN Person p ON a.athlete_code = p.person_code
WHERE pp.discipline LIKE '%Surfing%';
```

	ATHLETE_NAME
1	Alan CLELAND QUINONEZ
2	Camilla KEMP
3	Tim ELTER
4	Ramzi BOUKHIAM
5	Bryan PEREZ
6	Brisa HENNESSY
7	Johanne DEFAY
8	Vahine FIERRO
9	Kauli VAAST
10	Joan DURU
11	Alonso CORREA
12	Lucca MESINAS

Following a similar validation format as many other queries, to validate the total_Athletes count from the query, a specific discipline was taken and all the athletes names under such discipline were outputted. In this case, Surfing returned a total of 48 different athletes under such discipline, which is equal to the count from the question query.

7.21 Which 3 days in the schedule have the highest amount of scheduled events starting at the same time?

```
SELECT SUBSTR(s.start_date, 1, 10) AS event_day, SUBSTR(s.start_date, 12, 5) AS
start_time, COUNT(*) AS event_count
FROM Schedule s
GROUP BY event_day, start_time
ORDER BY event_count DESC
FETCH FIRST 3 ROWS ONLY;
```

	EVENT_DAY	START_TIME	EVENT_COUNT
1	2024-07-28	12:00	69
2	2024-07-29	12:00	54
3	2024-07-30	12:00	22

7.21.1 Explanation

This query extracts from the Schedule table the date and time components from each events timestamp and groups events by these two fields. Counting the rows allows us to reveal how many events start at the same time. Additionally, ordering results by this count will highlight the three time slots with the heaviest scheduling concentration, where the last line of the query will reveal the three time slots with most events. It's to be considered that the SUBSTR was used twice for start_date, since the date and time both are saved in the same attribute, allowing to extract both individually.

7.21.2 Validation

```
SELECT * FROM Schedule
WHERE SUBSTR(start_date, 1, 10) = '2024-07-30' AND SUBSTR(start_date, 12, 5) = '12:00';
```

	SCHEDULE_ID	DAY	STATUS	SCHEDULE_URL	LOCATION	VENUE	EVENT_ID	START_DATE	END_I
1	1266	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	BV0	EIF	18	2024-07-30T12:00...	2024-(
2	1267	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	JU1	CDM	28	2024-07-30T12:00...	2024-(
3	1268	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	TSL	RGA	40	2024-07-30T12:00...	2024-(
4	1269	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	T14	RGA	40	2024-07-30T12:00...	2024-(
5	1270	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	TSL	RGA	40	2024-07-30T12:00...	2024-(
6	1271	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	INV	INV	64	2024-07-30T12:00...	2024-(
7	1272	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	TSL	RGA	92	2024-07-30T12:00...	2024-(
8	1273	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	T14	RGA	92	2024-07-30T12:00...	2024-(
9	1274	30/07/2024, 12:00:...	FINISHED	/en/paris-2024/re...	T14	RGA	92	2024-07-30T12:00...	2024-(

The query was checked by printing out every single scheduled event for event day 2024-07-30, where a total of 22 rows with different scheduled events was outputted, verifying the validity of the event count.

7.22 Which athletes are currently living in a country different from the one they're officially representing?

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, r.residence_country, c.country
FROM Person p
JOIN Country c ON p.noc_code = c.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Lives l ON a.athlete_code = l.athlete_code
JOIN Residence r ON l.residence_ID = r.residence_ID
WHERE r.residence_country <> c.country
GROUP BY p.first_name, p.second_name, r.residence_country, c.country;
```

	ATHLETE_NAME	RESIDENCE_COUNT	COUNTRY
1	Chiamaka NNADO...	France	Nigeria
2	Achraf HAKIMI	France	Morocco
3	Sofia SJOBORG	Great Britain	Sweden
4	Edriel YULO Carlos	Japan	Philippines
5	Patricia FERRANDO	United States	Venezuela
6	Kristian BJOERNSEN	Denmark	Norway
7	Cindy SEMBER	United States	Great Britain
8	Tui LEILUA Eroni	New Zealand	Samoa
9	Julien ALFRED	United States	Saint Lucia
10	ASHER-SMITH Dina	United States	Great Britain
11	Ella ONOJUVWEV...	United States	Nigeria
12	Amin ALSALAMI ...	Germany	EOR
13	Angela MALESTEIN	Hungary	Netherlands
14	Zouhair TALBI	United States	Morocco

7.22.1 Explanation

The query identifies athletes and joins them through Person to find the country they officially represent in the Games. Additionally, such athletes are then joined to Residence through Lives to find the country they currently reside in. Comparing the residence country with the athletes NOC country allows to detect such athletes where the two differ. Filtering on this mismatch isolates the athletes whose living situation doesn't match with the country he represents, which may be a mismatch in nationalities or purely that the athlete lives elsewhere.

7.22.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, r.residence_country, c.country
FROM Person p
JOIN Country c ON p.noc_code = c.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Lives l ON a.athlete_code = l.athlete_code
JOIN Residence r ON l.residence_ID = r.residence_ID
WHERE p.second_name = 'HAKIMI' AND p.first_name = 'Achraf'
GROUP BY p.first_name, p.second_name, r.residence_country, c.country;
```

	ATHLETE_NAME	RESIDENCE_COUNT	COUNTRY
1	Achraf HAKIMI	France	Morocco

A validation query was produced to verify that the data above matches reality. The athlete chosen, Achraf Hakimi, represented Morocco in the football tournament in the Olympics. However, he plays for Paris Saint-Germain, a football club in France, which therefore means the results are accurate.

7.23 Which sports have had all their events concentrated in the fewest number of venues?

```
SELECT s.sport, COUNT(DISTINCT sch.venue) AS venue_count
FROM Sport s
JOIN Showcases sh ON s.sport_code = sh.sport_code
JOIN Event e ON sh.event_ID = e.event_ID
JOIN Schedule sch ON e.event_ID = sch.event_ID
GROUP BY s.sport
ORDER BY venue_count ASC;
```

	SPORT	VENUE_COUNT
1	Swimming	1
2	Shooting	1
3	Canoe Sprint	1
4	Sport Climbing	1
5	Equestrian	1
6	Rhythmic Gymna...	1
7	Artistic Swimming	1
8	Breaking	1
9	Marathon Swimm...	1
10	Weightlifting	1

7.23.1 Explanation

Each sport is linked to the events it includes and then to the scheduled instances of those events, therefore allowing to obtain all venue information necessary through Schedule. Grouping by sport ensures that the venue count is computed independently for each sport. The count of distinct venues from the Schedule table prevents multiple scheduled sessions of the same event to inflate the final count. Ordering in ascending order will return those sports which have had the fewest number of venues first and consecutively increasing.

7.23.2 Validation

```
SELECT sp.sport, v.venue_name
FROM Sport sp
JOIN Showcases swc ON sp.sport_code = swc.sport_code
JOIN Event e ON swc.event_ID = e.event_ID
JOIN Schedule sch ON e.event_ID = sch.event_ID
JOIN Venue v ON sch.venue = v.venue_code
WHERE sp.sport LIKE 'Swimming'
GROUP BY sp.sport, v.venue_name;
```

	SPORT	VENUE_NAME
1	Swimming	Paris La Defense Arena

To validate the venue count in the query, another validation query using the sport Swimming was used to verify if truly all swimming events were done in the same venue. The query returns all venue names that are linked to the mentioned sport and in this case, only one venue name was returned, the same result as the venue count in the query.

7.24 Which venues have hosted the highest number of different events within a single competition day?

```
SELECT v.venue_name, s.day, COUNT(DISTINCT s.event_ID) AS count_events
FROM Venue v
JOIN Schedule s ON v.venue_code = s.venue
GROUP BY v.venue_code, v.venue_name, s.day
HAVING count_events = (
        SELECT MAX(COUNT(DISTINCT s2.event_ID))
        FROM Schedule s2
        WHERE s2.venue = v.venue_code
        GROUP BY s2.day
)
ORDER BY v.venue_name, s.day;
```

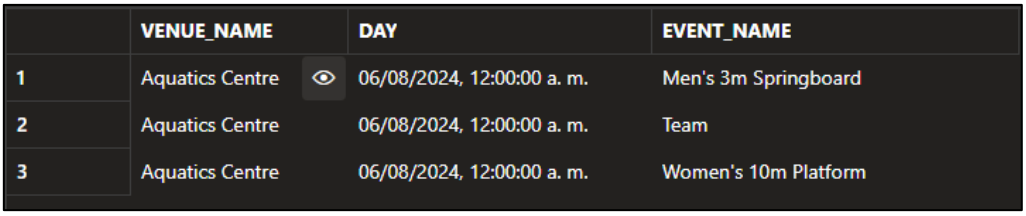
	VENUE_NAME	DAY	COUNT_EVENTS
1	Aquatics Centre	06/08/2024, 12:00:00 a. m.	3
2	Aquatics Centre	07/08/2024, 12:00:00 a. m.	3
3	Aquatics Centre	09/08/2024, 12:00:00 a. m.	3
4	Bercy Arena	05/08/2024, 12:00:00 a. m.	4
5	BMX Stadium	01/08/2024, 12:00:00 a. m.	2
6	BMX Stadium	02/08/2024, 12:00:00 a. m.	2
7	Bordeaux Stadium	24/07/2024, 12:00:00 a. m.	1
8	Bordeaux Stadium	25/07/2024, 12:00:00 a. m.	1
9	Bordeaux Stadium	27/07/2024, 12:00:00 a. m.	1
10	Bordeaux Stadium	30/07/2024, 12:00:00 a. m.	1
11	Bordeaux Stadium	31/07/2024, 12:00:00 a. m.	1
12	Bordeaux Stadium	02/08/2024, 12:00:00 a. m.	1

7.24.1 Explanation

The query links each venue to the events scheduled by linking Schedule with Venue. Grouping by both venue name and the competition day allows the logic to compute how many distinct events each venue hosted for a single day. Furthermore, a count of the distinct event ids was computed to ensure no repeated schedule entries inflated the final count. A subquery is performed in order to find the max count of distinct events that were scheduled in the venue within a single day, which is then compared with the current day.

7.24.2 Validation

```
SELECT v.venue_name, s.day, e.event_name
FROM Venue v
JOIN Schedule s ON v.venue_code = s.venue
JOIN Event e ON s.event_ID = e.event_ID
WHERE v.venue_name = 'Aquatics Centre' AND s.day = TO_DATE('06/08/2024 00:00',
'DD/MM/YYYY HH24:MI')
ORDER BY e.event_name;
```



	VENUE_NAME	DAY	EVENT_NAME
1	Aquatics Centre	06/08/2024, 12:00:00 a. m.	Men's 3m Springboard
2	Aquatics Centre	06/08/2024, 12:00:00 a. m.	Team
3	Aquatics Centre	06/08/2024, 12:00:00 a. m.	Women's 10m Platform

Event_count was validated by taking one of the venues names outputted and all its distinct events were outputted in another query. Specifically, by taking the venue Aquatics Centre, all its different events on the 06/08/2024 were output, meaning the count was correct. Specifically, this also proves that the same venue may have multiple days that have the maximum number of events scheduled on the same day.

7.25 Which countries have the largest number of athletes competing but haven't won any medals so far?

```
SELECT c.country, COUNT(a.athlete_code) AS total_athletes
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
WHERE c.noc_code NOT IN (
SELECT DISTINCT c3.noc_code
FROM Country c3
JOIN Person p2 ON c3.noc_code = p2.noc_code
JOIN Athlete a2 ON p2.person_code = a2.athlete_code
JOIN Athlete_wins aw ON a2.athlete_code = aw.athlete_code
WHERE aw.is_medallist = 'True'
)
GROUP BY c.country
ORDER BY total_athletes DESC;
```

	COUNTRY	TOTAL_ATHLETES
1	Nigeria	86
2	Finland	57
3	Venezuela	31
4	Latvia	29
5	Paraguay	29
6	Uruguay	27
7	Guinea	25
8	Angola	25
9	Mali	24
10	Samoa	24
11	Estonia	24

7.25.1 Explanation

For each country, the query links its athletes through the Person and Athlete tables, allowing a count of all competitors that represent that nation. A combined exclusion list is built in a subquery that identifies every country that has earned at least one medal in the Olympic games through team results or through individual athlete performances. Using this list inside the NOT IN filter allows to remove all countries that have achieved any medal. Grouping by country aggregates the number of athletes for each remaining delegation. Lastly, by sorting with the athlete count, this highlights the largest non-medaling countries in the competition.

7.25.2 Validation

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name, aw.is_medallist
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
LEFT JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE c.country = 'Venezuela'
ORDER BY athlete_name;
```

	ATHLETE_NAME	IS_MEDALLIST
1	Alberto MESTRE	*(nulo)*
2	Andreina RODRIG...	*(nulo)*
3	Anriquelis BARRIOS	*(nulo)*
4	Antoni CARABALL...	*(nulo)*
5	Anyelin VENEGAS ...	*(nulo)*
6	Betzabeth Angeli ...	*(nulo)*
7	Carolina ALCALA ...	*(nulo)*
8	Daniely BREA Jose...	*(nulo)*
9	Douglas GOMEZ	*(nulo)*
10	Enrique MESTRE V...	*(nulo)*
11	F PAREDES TORRE...	*(nulo)*

To assess the results of 114 countries' output that have no medalists, one of those countries was taken and all its athletes were output. Specifically, Venezuelan athletes were outputted, where these were linked with a left join to the athlete_wins table, since theoretically it would return null since they've never won medals. This is confirmed as for all Venezuelan athletes, the is_medallist attribute is null, confirming Venezuela has won zero medals.

7.26 Which countries have won more team gold medals than the combined total of their team silver and bronze medals?

```
SELECT c.country,
    SUM(CASE WHEN m.medal_type = 'Gold Medal' THEN 1 ELSE 0 END) AS gold_count,
    SUM(CASE WHEN m.medal_type IN ('Silver Medal', 'Bronze Medal') THEN 1 ELSE 0
END) AS silver_bronze
FROM Country c
JOIN Team_represents tr ON c.noc_code = tr.noc_code
JOIN Team t ON tr.team_code = t.team_code
JOIN Team_wins tw ON t.team_code = tw.team_code
JOIN Medal m ON tw.medal_ID = m.medal_ID
WHERE tw.is_medallist = 'True'
GROUP BY c.country
HAVING SUM(
        CASE WHEN m.medal_type = 'Gold Medal' THEN 1 ELSE 0 END) >
        SUM(CASE WHEN m.medal_type IN ('Silver Medal', 'Bronze Medal') THEN 1 ELSE 0
END
)
ORDER BY gold_count DESC;
```

	COUNTRY	GOLD_COUNT	SILVER_BRONZE
1	China	16	11
2	Netherlands	9	7
3	Serbia	2	1
4	Chinese Taipei	1	0
5	Austria	1	0
6	Portugal	1	0

7.26.1 Explanation

This query links each country to the teams that represent it and then connects it to the medals those teams have earned through the Team_wins table. Aggregating medal types with conditional SUM operators and cases allows us to count separate gold medals from silver and bronze medals. Grouping by country ensures that all team results belonging to the same nation are combined into one single entry. The HAVING clause filters the output to only showcase those countries that have more golden medals than both silver and bronze combined. Ordering by the gold count highlights the strongest team performances for earned golden medals.

7.26.2 Validation

```
SELECT c.country, SUM(CASE WHEN m.medal_type = 'Gold Medal'   THEN 1 ELSE 0 END)
AS gold, SUM(CASE WHEN m.medal_type = 'Silver Medal' THEN 1 ELSE 0 END) AS silver,
SUM(CASE WHEN m.medal_type = 'Bronze Medal' THEN 1 ELSE 0 END) AS bronze
FROM Country c
JOIN Team_represents tr ON c.noc_code = tr.noc_code
JOIN Team_wins tw ON tr.team_code = tw.team_code
JOIN Medal m ON tw.medal_ID = m.medal_ID
WHERE tw.is_medallist = 'True' AND c.country = 'United States'
GROUP BY c.country;
```

	COUNTRY	GOLD	SILVER	BRONZE
1	United States	13	12	9

The validity of the previous query is proved by taking another country from those not output and checking if it was correctly filtered previously. For the case of the United States, their total of silver and bronze medals is greater than their total gold medals in team competitions, demonstrating that it was correctly filtered and the results from the previous query are correct.

7.27 Which countries have both individual medalists and team medalists?

```
SELECT DISTINCT c1.country
FROM Country c1
JOIN Person p ON c1.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE aw.is_medallist = 'True'

INTERSECT

SELECT DISTINCT c2.country
FROM Country c2
JOIN Person p ON c2.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Members m ON a.athlete_code = m.athlete_code
JOIN Team t ON m.team_code = t.team_code
JOIN Team_wins tw ON t.team_code = tw.team_code
WHERE tw.is_medallist = 'True';
```

	COUNTRY
1	Argentina
2	India
3	Italy
4	France
5	Great Britain
6	Germany
7	Croatia
8	Morocco
9	Denmark
10	Poland
11	Norway
12	South Africa
13	Spain

7.27.1 Explanation

The first query isolates individual athletes who have earned medals for their respective countries by joining their records with the medal tables. A second query identifies athletes who have also earned medals but as a part of a team, by following membership and team-medal tables, restricting results by team-based winnings. Using INTERSECT ensures that only countries who have won medals in both results are returned. It must be considered that for the table Athlete, both individual and team medalists are included, where on the other hand, a Team has only athletes who won medals.

7.27.2 Validation

```
SELECT COUNT(DISTINCT a.athlete_code) AS Number_Athletes
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE c.country = 'Italy' AND aw.is_medallist = 'True';
```

	NUMBER_ATHLETE
1	80

```
SELECT COUNT(DISTINCT a.athlete_code) AS Number_Athletes
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Members m ON a.athlete_code = m.athlete_code
JOIN Team t ON m.team_code = t.team_code
JOIN Team_wins tw ON t.team_code = tw.team_code
WHERE c.country = 'Italy' AND tw.is_medallist = 'True';
```

	NUMBER_ATHLETE
1	13

Firstly, it must be proven that the query was considering both individual athlete wins, as well as team wins. Therefore, two different counts were computed for Team wins medals and Athlete wins medals. This means that a total of 13 medals of the total 80 are team medals, whereas the lasting 67 medals are individual medals. Therefore, this means that Italy does have medals in both blocks.

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE c.country = 'Italy' AND aw.is_medallist = 'True'
GROUP BY athlete_name
ORDER BY athlete_name DESC;
```

	ATHLETE_NAME
1	Ogechi EGONU Pa...
2	Monica de GENN...
3	Marina LUBIAN
4	Luisa FAHR Sarah
5	Loveth OMORUYI ...
6	Ilaria SPIRITO
7	Gaia GIOVANNINI
8	Fatime SYLLA Myri...
9	Ekaterina ANTROP...
10	Chiara BOSETTI Ca...
11	Carlotta CAMBI
12	Anna DANESI
13	Alessia ORRO

```
SELECT p.first_name || ' ' || p.second_name AS athlete_name
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Members m ON a.athlete_code = m.athlete_code
JOIN Team t ON m.team_code = t.team_code
JOIN Team_wins tw ON t.team_code = tw.team_code
WHERE c.country = 'Italy' AND tw.is_medallist = 'True'
GROUP BY athlete_name
ORDER BY athlete_name DESC;
```

	ATHLETE_NAME
1	Vittoria GUAZZINI
2	Tommaso MARINI
3	Thomas CECCON
4	Stefano OPPO
5	Sofia RAFFAELI
6	Simone CONSONNI
7	Simone ALESSIO
8	Sara ERRANI
9	Ruggero TITA
10	Rossella FIAMINGO
11	Paolo MONNA
12	Paolo CONTE BO...
13	Ogechi EGONU Pa...

Additionally, to prove that the count was correctly distinguishing between different athletes, two different validation queries were computed to output all athlete names who have won or team medals or athlete medals. In this case, 13 rows of athletes winning medals with a team were output, which coincides with the previous result. For the other query as well, there were a total of 80 winning athletes, including the team winning medalists. This can be proved since in both images above, the athlete Ogechi Egonu appears for both team and athlete, meaning a team medalist.

7.28 Which sport is the most trained among athletes that have won a medal?

```
SELECT sp.sport, COUNT(DISTINCT tns.training_ID) AS total_sessions
FROM Athlete a
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Practices prc ON a.athlete_code = prc.athlete_code
JOIN Sport sp ON prc.sport_code = sp.sport_code
JOIN Trains tns ON a.athlete_code = tns.athlete_code
WHERE aw.is_medallist = 'True'
GROUP BY sp.sport, sp.sport_code
ORDER BY total_sessions DESC
FETCH FIRST 1 ROWS ONLY;
```

	SPORT	TOTAL_SESSIONS
1	Athletics	1178

7.28.1 Explanation

Medal-winning athletes are linked to their training activity by joining their medal records with both their Trains and Training tables, guaranteeing the correct session and information are collected. Additionally, we connect the sport the athlete competes in through table Practices, allowing us to attribute all training activities of such athletes to that sport. Grouping by sport allows to aggregate the total number of training sessions completed by medallists. Results are ordered in descending order to

surface the sport with the highest accumulated number of training sessions, which the first row with the greatest overall training will be outputted due to the FETCH FIRST 1 ROWS ONLY.

7.28.2 Validation

```
SELECT sp.sport, COUNT(DISTINCT tns.training_ID) AS total_sessions
FROM Athlete a
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Practices prc ON a.athlete_code = prc.athlete_code
JOIN Sport sp ON prc.sport_code = sp.sport_code
JOIN Trains tns ON a.athlete_code = tns.athlete_code
WHERE aw.is_medallist = 'True'
GROUP BY sp.sport, sp.sport_code
ORDER BY total_sessions DESC
FETCH FIRST 5 ROWS ONLY;
```

	SPORT	TOTAL_SESSIONS
1	Athletics	1178
2	Swimming	1023
3	Rowing	903
4	Football	805
5	Hockey	635

Firstly, the query was validated by checking if truly Athletics was the sport with the most training sessions. By also printing the total sessions count for other sports, it's demonstrated that truly Athletics has the greatest count out of all sports.

```
SELECT sp.sport, tns.training_id
FROM Athlete a
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
JOIN Practices prc ON a.athlete_code = prc.athlete_code
JOIN Sport sp ON prc.sport_code = sp.sport_code
JOIN Trains tns ON a.athlete_code = tns.athlete_code
WHERE aw.is_medallist = 'True' AND sp.sport = 'Athletics'
GROUP BY sp.sport, tns.training_id;
```

657	Athletics	45943
658	Athletics	24474
659	Athletics	30488
660	Athletics	60868
661	Athletics	39243
662	Athletics	44706
663	Athletics	57921
664	Athletics	64866
665	Athletics	6186
666	Athletics	30251

Secondly, another validation query was performed to guarantee that truly there were 1178 different unique training sessions for medalists in the sport Athletics. Therefore, all training sessions for medalists were outputted, where a total of 1178 rows were returned, truly demonstrating the validity of the count of training sessions of medalists in the sport Athletics.

7.29 Which venues have had at least one scheduled event on both the first and the last day of the Olympics?

```
SELECT v.venue_name
FROM Venue v
JOIN Schedule s ON v.venue_code = s.venue
WHERE TRUNC(s.day) = (SELECT MIN(TRUNC(day)) FROM Schedule)

INTERSECT

SELECT v.venue_name
FROM Venue v
JOIN Schedule s ON v.venue_code = s.venue
WHERE TRUNC(s.day) = (SELECT MAX(TRUNC(day)) FROM Schedule);
```

VENUE_NAME
No se ha encontrado ningún dato

7.29.1 Explanation

To output the venues who have had at least one scheduled event on the first and last day of the Olympics, an intersect was implemented. Both venue and schedule were linked through JOIN to connect those scheduled events in each specific venue. Additionally, WHERE clauses were used in both small queries to compare the day of the current venue to the MAX and MIN of day, which are the first and last day of the Olympics. TRUNC was utilized to remove the time part from the day, to be able to only compare the day without any time constraints. Therefore, the intersection with both

small blocks, will allow to only output those venues who have had a scheduled event on both the first and last day of the Olympics.

7.29.2 Validation

```
SELECT DISTINCT v.venue_name
FROM Venue v
JOIN Schedule s ON v.venue_code = s.venue
WHERE TRUNC(s.day) = (SELECT MIN(TRUNC(day)) FROM Schedule);
```

	VENUE_NAME
1	Bordeaux Stadium
2	Lyon Stadium
3	Marseille Stadium
4	La Beaujoire Stadi...
5	Nice Stadium
6	Parc des Princes
7	Stade de France
8	Geoffroy-Guichar...

```
SELECT DISTINCT v.venue_name
FROM Venue v
JOIN Schedule s ON v.venue_code = s.venue
WHERE TRUNC(s.day) = (SELECT MAX(TRUNC(day)) FROM Schedule);
```

	VENUE_NAME
1	Bercy Arena
2	Champ-de-Mars A...
3	Paris La Defense A...
4	Invalides
5	Pierre Mauroy Sta...
6	South Paris Arena 1
7	South Paris Arena 6
8	National Velodrome
9	Château de Versail...

Since the query returned zero venues that had an event scheduled on both those days, to validate the query, both blocks that were linked with an INTERSECT, were executed individually, where one would return venues for the first day and the other for the last day. Both results demonstrate that none

of the venue names coincide, meaning that the original query was valid, proving no venues had an event scheduled for the first and last day of the Olympics.

7.30 Which are the top 3 countries with the highest number of athlete medallists whose training sessions have an intensity level above 7?

```
SELECT c.country, COUNT(DISTINCT a.athlete_code) AS medallist_count
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Trains trs ON a.athlete_code = trs.athlete_code
JOIN Training tng ON trs.training_ID = tng.training_ID
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE tng.intensity_level > 7 AND aw.is_medallist = 'True'
GROUP BY c.country
ORDER BY medallist_count DESC
FETCH FIRST 3 ROWS ONLY;
```

	COUNTRY	MEDALLIST_COUN
1	United States	222
2	France	146
3	Great Britain	121

7.30.1 Explanation

This query identifies, through multiple JOIN operations, it connects the Country, Person, Athlete, Training and Medal tables to properly link each medallist to the training session they have completed. Filtering on training intensity ensures that only athletes with an intensity higher than seven contribute to the final counts. Group by country allows to aggregate the number of distinct medallists to meet the criteria necessary for each nation. Lastly, the query limits the output to the top three countries with the strongest combination of number of medallists and high intensity training.

7.30.2 Validation

```
SELECT c.country, p.first_name || ' ' || p.second_name AS athlete_name
FROM Country c
JOIN Person p ON c.noc_code = p.noc_code
JOIN Athlete a ON p.person_code = a.athlete_code
JOIN Trains trs ON a.athlete_code = trs.athlete_code
JOIN Training tng ON trs.training_ID = tng.training_ID
JOIN Athlete_wins aw ON a.athlete_code = aw.athlete_code
WHERE tng.intensity_level > 7 AND aw.is_medallist = 'True' AND c.country = 'United States'
GROUP BY c.country, p.first_name, p.second_name;
```

	COUNTRY	ATHLETE_NAME
1	United States	Hampton MORRIS
2	United States	WONG-ORANTES ...
3	United States	MA'A Micah
4	United States	Micah CHRISTENS...
5	United States	Jeffrey JENDRYK II
6	United States	William DODD Ch...
7	United States	Max IRVING
8	United States	Jaime CZARKOWSKI
9	United States	Jagger EATON
10	United States	Sophia SMITH
11	United States	Lynn WILLIAMS
12	United States	Liam CORRIGAN
13	United States	Hunter ARMSTRO...
14	United States	Bobby FINKE
15	United States	Jordan CHILES

Lastly, the query was validated by verifying if the medalist count for one of the countries obtained in the query was correct. All medalists' names for the United States who have an intensity level of training higher than 7 was returned, which came to a total of 222 rows, proving the count was calculated correctly.

8 Trigger Implementation

8.1 Trigger 1

8.1.1 Introduction

Our idea for this trigger was inspired by the repeated validation and auditing examples seen in class, where triggers are used not only to enforce database rules, but also to monitor and preserve data quality automatically. In this case, the trigger addresses a data integrity concern in the Training table related to missing training dates. Within the context of the Olympic Games, training schedules are a fundamental part of athlete preparation, since coaches, medical staff, and performance analysts rely on precise dates to organize sessions, evaluate progress, and coordinate competition readiness. A training session without a date loses much of its practical value because it cannot be properly tracked, referenced, compared, or included in long-term planning and performance analysis.

Instead of completely rejecting the insertion or update of these records, we decided to implement a softer validation mechanism through a trigger-based warning system. This approach allows the database to remain flexible while still identifying potentially problematic data that may require later review. Whenever a row is inserted or updated with a NULL value in the *training_date* column, the trigger automatically records a warning in a dedicated WARNING table. This table acts as a history that stores information about incomplete or suspicious records, making it easier for administrators, coaches, or database managers to identify inconsistencies and correct them when necessary.

8.1.2 Solution

```
DROP TABLE WARNING;

CREATE TABLE WARNING (
   warning_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,
   training_ID NUMBER,
   message VARCHAR2(200),
   current_time TIMESTAMP,
   PRIMARY KEY (warning_ID)
);


DROP TRIGGER warning_null_training_date;

CREATE OR REPLACE TRIGGER warning_null_training_date
BEFORE INSERT OR UPDATE ON Training
FOR EACH ROW
BEGIN
   IF :NEW.training_date IS NULL THEN
      INSERT INTO WARNING(training_ID, message, current_time)
      VALUES (:NEW.training_id, 'Training date is NULL', SYSTIMESTAMP);
   END IF;
END;
/
```

8.1.3 Explanation

The trigger fires before any INSERT or UPDATE on the Training table, and it works for each row. Inside the body, it checks whether the new value of training_date, which is known by :NEW.training_date, is NULL.

If that condition holds, a new record is inserted into the WARNING table containing the affected training_id, a descriptive message, and the exact timestamp of the operation via SYSTIMESTAMP, this would kelp to keep track of the login of that training session.

Importantly, the trigger does not prevent the operation from completing since the row is still inserted into Training, the idea is just to keep track on those trainings in which a date was not recorded. Furthermore, the WARNING table uses an identity column so that each warning entry gets a unique ID automatically, requiring no manual sequence management.

8.1.4 Validation

To verify the trigger behaves as intended, the following test was performed:

1. INSERT with NULL training_date

```
DELETE FROM Training WHERE training_ID = 3000000;
DELETE FROM Training WHERE training_ID = 3000001;

INSERT INTO Training (
   training_ID,
   training_date,
   duration,
   intensity_level
)
VALUES (
   3000000,
   NULL,
   45,
   5
);
```

We firstly insert a new Training session that has a NULL Date. However, since the training is made up and we are told in the statement that the data inserted in tables must be the ones from the CSV, we firstly ensure there isn't a previous row with the equivalent ID, because of a previous validation of the trigger. We do so by dropping possible inserts make during the validation process.

2. INSERT with valid training_date

```
INSERT INTO Training (
   training_ID,
   training_date,
   duration,
   intensity_level
)
VALUES (
   3000001,
   TO_DATE('29/08/2006','DD/MM/YYYY'),
   60,
   4
);
```

A follow-up test inserting a row with a valid training_date was also run, just to ensure no new warning were being logged in that case, which would confirm the trigger only fires when the condition is met.

3. Validate in Warning table

```
SELECT * FROM WARNING;
```

Finally, the only last checkup is to see whether the warning was inserted into its correspondent table. The fact that the INSERT into Training completes without error confirms the trigger does not block the operation.

	WARNING_ID	TRAINING_ID	MESSAGE	CURRENT_TIM
1	1	3000000	Training date is NULL	15/05/2026, ...

The presence of the corresponding row in WARNING confirms the logging logic executed correctly, only when the date inserted was a NULL value.

8.2 Trigger 2

8.2.1 Introduction

This trigger was designed to automate the monitoring of athlete activity and ensure that the information stored in the database accurately reflects the current training situation of each athlete.

Our idea behind the trigger comes from the importance of tracking athlete participation and performance in high-level sports environments such as the Olympic Games, where training intensity and consistency are key indicators of whether an athlete is actively competing and preparing for events. Maintaining this information manually would be inefficient and error-prone, especially considering the large number of athletes and training sessions stored in the database.

This trigger monitors the training load of athletes and automatically updates their status in the Person table. Specifically, if an athlete has accumulated three or more training sessions with an *intensity*_level above 6, the system considers them actively engaged in high-performance training and sets their *is_current* flag to 'True'. This avoids the need to manually track athlete activity and keeps the status field consistent with actual training records.

8.2.2 Solution

```
DROP TRIGGER athlete_current_training;

CREATE OR REPLACE TRIGGER athlete_current_training
AFTER INSERT ON Trains
DECLARE
   v_high_count NUMBER;
BEGIN
  FOR r IN (
     SELECT DISTINCT athlete_code
     FROM Trains
     WHERE athlete_code IN (SELECT athlete_code FROM Trains)
   )
   LOOP
     SELECT COUNT(*)
     INTO v_high_count
     FROM Trains t
     JOIN Training tr ON tr.training_ID = t.training_ID
     WHERE t.athlete_code = r.athlete_code AND tr.intensity_level > 6;

     IF v_high_count >= 3 THEN
        UPDATE Person
        SET is_current = 'True'
        WHERE person_code = r.athlete_code;
     END IF;

  END LOOP;

  DBMS_OUTPUT.PUT_LINE('High intensity count: ' || v_high_count); --validation purposes

END;
/
```

8.2.3 Explanation

The trigger fires AFTER an INSERT on the Trains table, which records the relationship between athletes and their training sessions. Because it is a statement-level trigger, so not applied FOR EACH ROW, it executes once per insert operation and reviews all athletes comprehensively. It loops through every distinct *athlete_code* present in Trains, and for each athlete it counts how many of their training sessions have an *intensity_level* greater than 6, which is done via a JOIN between Trains and Training. If that count reaches 3 or more, the athlete's record in Person is updated to *is_current* = 'True', since we believe training at a certain level mean you’re practicing for a competition.

Also, the DBMS_OUTPUT line is included for debugging purposes during development, allowing us to inspect the final count value after the loop completes.

8.2.4 Validation

The validation resets the target athlete's status and removes any conflicting records before testing:

```
UPDATE Person
SET is_current = 'False'
WHERE person_code = 1878715;

DELETE FROM Trains
WHERE training_ID = 100000
AND athlete_code = 1878715;

DELETE FROM Training
WHERE training_ID = 100000;
```

1. INSERT triggering status update, for a new training session

```
INSERT INTO Training (training_ID, training_date, duration, intensity_level)
VALUES (100000, SYSDATE, 60, 8);
```

We ensure we insert a new training session in which the intensity level is high enough to fire the trigger.

2. INSERT complimentary to training session in table Trains

```
INSERT INTO Trains (athlete_code, training_ID)
VALUES (1878715, 100000);
```

We also insert a complimentary row to training session, which is in Trains, to link an athlete to that inserted training session. For our validation, we will keep track of the athlete with the ID of 1878715.

3. Validate Update

```
SELECT a.athlete_code, p.is_current
FROM Person p
JOIN Athlete a ON a.athlete_code = p.person_code
WHERE person_code = 1878715;
```

The variable is_current is set to 'True' for athlete 1878715, since this athlete already had at least two prior high-intensity sessions and this new one (intensity 8 > 6) pushes the count to 3 or more.

	ATHLETE_CODE	IS_CURRENT
1	1878715	True

The query on Person confirms the update was applied. After validation, all test data is cleaned up to restore the original state, since the statement specifically asks for the CSV imported data to remain the same.

8.3 Trigger 3

8.3.1 Introduction

The purpose of this trigger is to automatically maintain a historical record of medal achievements within the Olympic Games database. In large sports management systems, it is important not only to store the direct relationship between athletes and medals, but also to preserve a readable and centralized history of these accomplishments for reporting, analysis, and auditing purposes. Every time an athlete wins a medal, that achievement becomes a significant event that should be permanently recorded in a format that is easy to interpret and query without requiring multiple table joins.

To achieve this, our idea for this trigger was to activate it whenever a new row is inserted into the Athlete_wins table. Instead of relying on manual updates to a history table, the trigger automatically creates a corresponding entry in the Medal_History table. Additionally, the trigger would make a simplified medal category label, GOLD, SILVER, BRONZE, or NONE based on the medal information, since we had noticed the medal category was stored with integers.

8.3.2 Solution

```
CREATE OR REPLACE TRIGGER history_medal_win
AFTER INSERT ON Athlete_wins
FOR EACH ROW
DECLARE
  v_first_name VARCHAR2(255);
  v_second_name VARCHAR2(255);
  v_medal_type VARCHAR2(25);
  v_label VARCHAR2(10);
BEGIN
   SELECT p.first_name, p.second_name
   INTO   v_first_name, v_second_name
   FROM   Person p
   WHERE  p.person_code = :NEW.athlete_code;

   SELECT m.medal_type
   INTO v_medal_type
   FROM Medal m
   WHERE m.medal_ID = :NEW.medal_ID;

   CASE
      WHEN v_medal_type = 'Gold Medal' THEN v_label := 'GOLD';
      WHEN v_medal_type = 'Silver Medal' THEN v_label := 'SILVER';
      WHEN v_medal_type = 'Bronze Medal' THEN v_label := 'BRONZE';
      ELSE v_label := 'NONE';
   END CASE;

   IF :NEW.is_medallist = 'True' THEN
      INSERT INTO Medal_History(
         athlete_code,
         first_name,
         second_name,
         medal_type,
         medal_label,
         is_medallist,
         log_time
      )
      VALUES(
         :NEW.athlete_code,
         v_first_name,
         v_second_name,
         v_medal_type,
         v_label,
         :NEW.is_medallist,
         SYSDATE
      );
```

```
ELSE
      INSERT INTO Medal_History(
         athlete_code,
         first_name,
         second_name,
         medal_type,
         medal_label,
         is_medallist,
         log_time
      )
      VALUES(
         :NEW.athlete_code,
         v_first_name,
         v_second_name,
         v_medal_type,
         'NONE',
         :NEW.is_medallist,
         SYSDATE
      );
   END IF;
END;
/
```

8.3.3 Explanation

The trigger fires AFTER each row is inserted into Athlete_wins. It first performs two SELECT INTO queries: one to retrieve the athlete's first_name and second_name from the Person table, and one to retrieve the medal_type from the Medal table, both using the foreign key values present in :NEW. A CASE block then maps the full medal type string to a concise label.

Finally, the trigger checks the is_medallist flag, if it is 'True', the actual medal label is stored in Medal_History, otherwise, the label is forced to 'NONE' regardless of the medal type, reflecting the fact that non-medallists, should not be credited with a medal. The history_id sequence ensures each history entry has a unique, auto-incremented primary key.

8.3.4 Validation

Before testing, any conflicting records are removed to ensure a clean state:

```
DELETE FROM Medal_History
WHERE athlete_code = 1566564
AND medal_type = 'Gold Medal';

DELETE FROM Athlete_wins
WHERE medal_ID = 99999
AND athlete_code = 1566564;

DELETE FROM Medal
WHERE medal_ID = 99999;

DROP TRIGGER history_medal_win;

DROP TABLE Medal_History;
```

1. INSERT before the trigger:

```
INSERT INTO Medal(medal_ID, medal_type, medal_code, medal_date)
VALUES (99999, 'Gold Medal', 999, SYSDATE);
```

Firstly, we insert an invented medal that we can reference an athlete to, so that it doesn't affect its attributes.

2. INSERT triggering a log in history:

```
INSERT INTO Athlete_wins (medal_ID, athlete_code, is_medallist)
VALUES (99999, 1566564, 'True');
```

Then we make an insert so that the trigger will work, adding a new medal to an athlete's collection.

3. Validate trigger in Medal History

```
SELECT *
FROM Medal_History
ORDER BY history_id DESC
FETCH FIRST 1 ROW ONLY;
```

The most recent row in Medal_History should show medal_label = 'GOLD' and is_medallist = 'True' for athlete 1566564. This confirms that both the CASE label derivation and the is_medallist branching logic are working correctly.

HISTORY_ID	ATHLETE_CODE	FIRST_NAME	SECOND_NAME	MEDAL_TYPE	MEDAL_LABEL	IS_MEDALLIST
1	1566564	Aaron	CHIA	Gold Medal	GOLD	True

9 Conclusions

9.1 Utilization of resources (1 page)

Phase	Sara Gibert	Alex Vilamitjana	Oriol Pros	Total
Entity-Relationship model	4h	2h	1h	7h
Relational model	1h	3h	1h	5h
Datatype selection	1h	3h	3h	7h
Physical model codification	2h	5h	4h	11h
Data import	9h	2h	3h	14h
Query and trigger implementation	7h	7h	6h	20h
Report	4h	5h	8h	17h
Total:	28h	27h	26h	81h

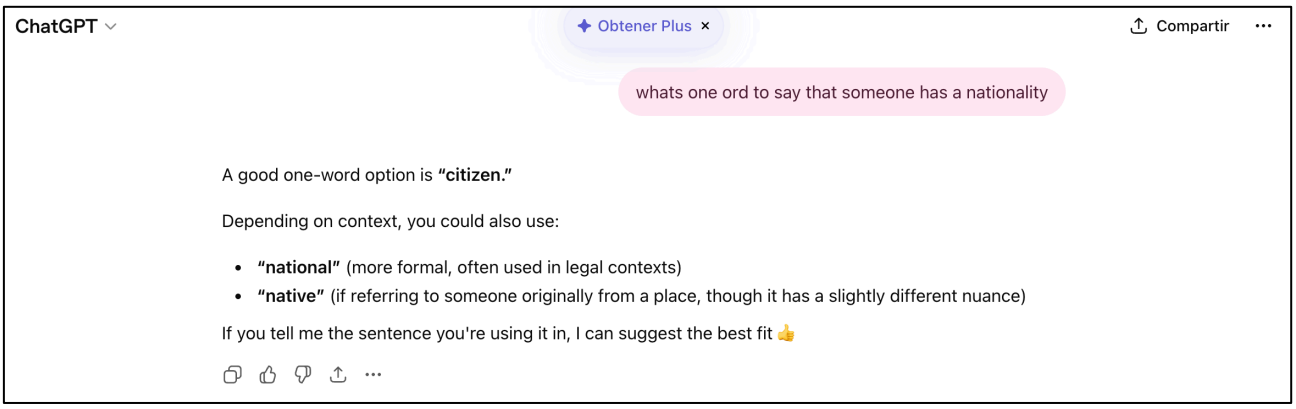
The project took 81 hours of work distributed fairly evenly across the three team members: Sara Gibert (28h), Alex Vilamitjana (27h), and Oriol Pros (26h). The most time-consuming phases were query and trigger implementation (20h), the report (17h), and data import (14h), reflecting the inherent complexity of translating the Olympic Games 2024 dataset into a functioning, well-queried relational database.

The most notable deviations from an even split occurred in the data import phase, where Sara contributed a disproportionate 9 hours compared to Alex's 2h and Oriol's 3h, due to the challenges of cleaning and loading the nine heterogeneous CSV files. In the physical model codification phase, Alex and Oriol led the effort (5h and 4h respectively) while Sara contributed less (2h), as she had already invested more time in the earlier entity-relationship modelling. Similarly, the report writing was primarily driven by Oriol (8h), who took on the bulk of the writing and structuring, while Sara and Alex contributed 4h and 5h respectively. These deviations reflect a natural task-based division of labour rather than imbalanced overall contribution, as the final totals across all phases remain well balanced.

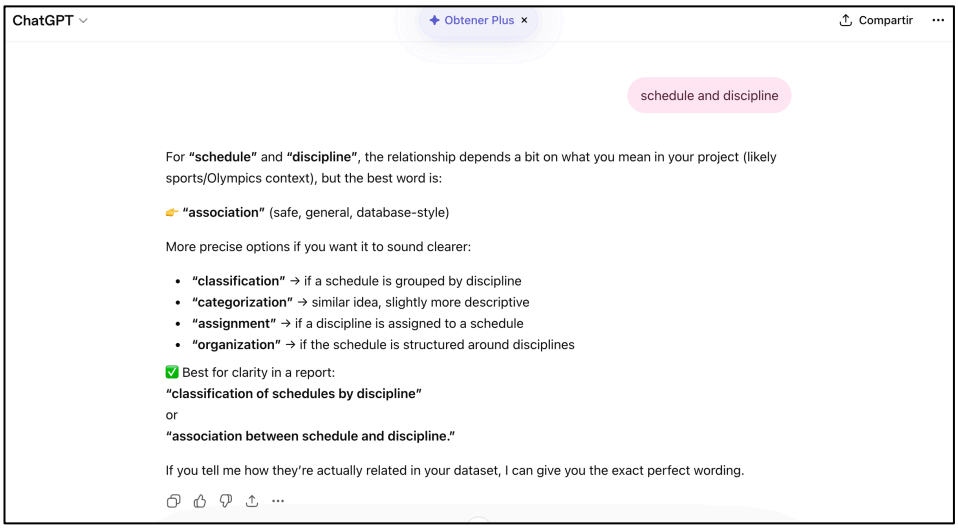
9.2 AI usage (if needed, 1-2 pages)

In this project, we had expected to use AI when trying to simplify entities or solving doubts, but we ended up using it in very few occasions.

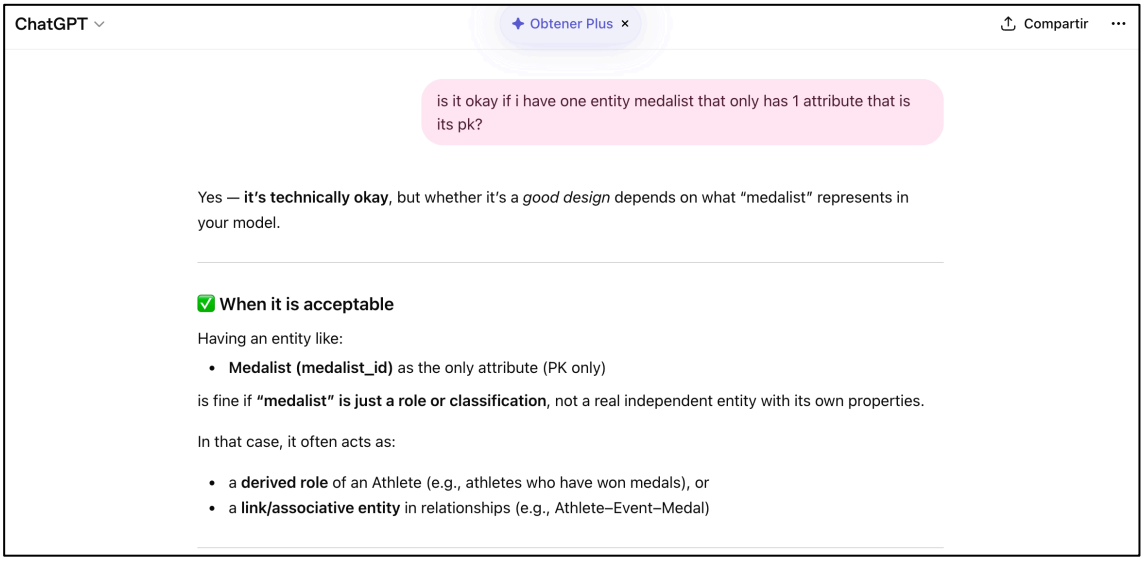
Mainly, it was to look up for specific words to represent better the relationships. For example, the relationship between person and nationality, we would have written *"has"*, but ChatGPT suggested *"Citizen"*. Like so, we wanted to ensure that the words chosen to represent each relationship was the most accurate as possible, so that there wouldn't be confusion when further developing the model. Here are some of the examples we used:

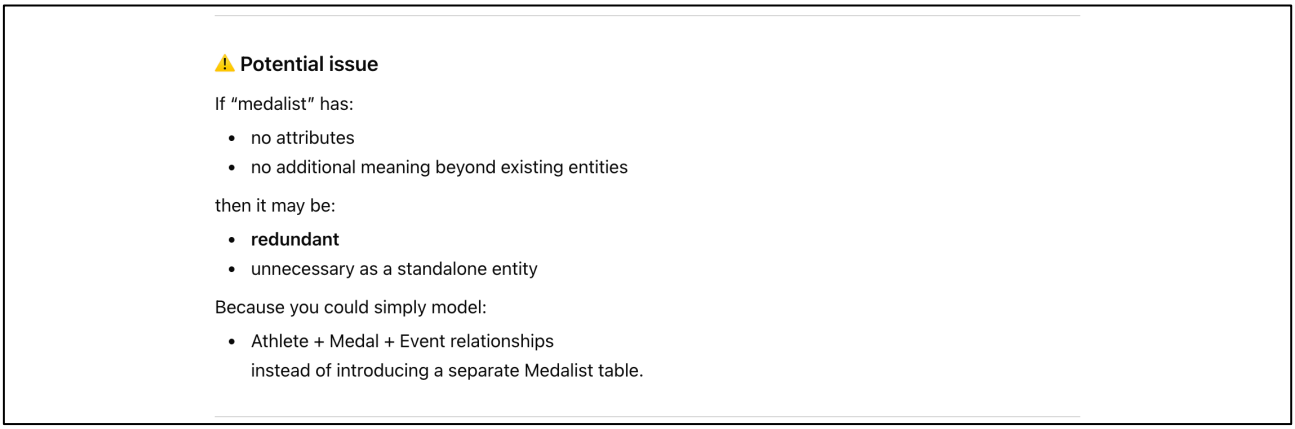






Also, another consult we made, was a doubt we had, and had been talking over it for a large amount of time, which was whether to include an entity named *“Medalist”*, following the idea of creating one entity per *csv* file, or directly ignore it since we could simplify it with the already created entity *“Medal”*.



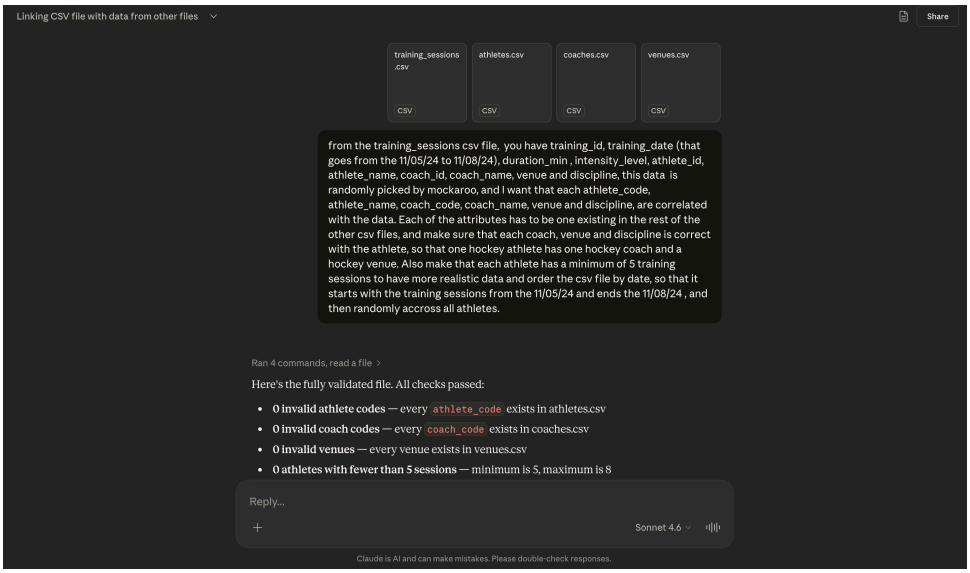


With this analogy, we ended up deciding to remove the entity medalist, as explained previously in the correspondent part of the report. It was indeed redundant, and thus, not necessary.

Also, while creating the new CSV file, *training_sessions*, with the tool mockaroo, we couldn't get the file to have values from other CSV files, so in order to do that we asked Claude to update the original CSV file with values that already exist and are in the files given to us such as *athletes.csv*, *coaches.csv*, and *venues.csv*.

We wanted the new changed file to have some considerations, so on the following phases, the data could be more organized and better represented. In *figure 8* and *figure 9* there is the prompt sent to Claude that updated the CSV with the following considerations:

- *Athlete_id, coach_id, discipline* must have existing values from other CSV files.
- All the data is correlated with the data, so one Hockey athlete, has one Hockey coach, at one Hockey venue.
- Each athlete must have a minimum of 5 training sessions.
- Ordered by date.



After this prompt, we reviewed the CSV file to ensure everything was correct, and it was indeed, all the athletes, coaches, venues and disciplines information was related to each other and therefore, the new CSV file was created with the considerations and relation with existing entries.

9.3 Lessons learned and conclusions (1 page)

The development of this database project around the Paris 2024 Olympic Games has been one of the most comprehensive and demanding exercises of the Databases course. Working with a rich, multi-entity real-world dataset, such as spanning athletes, coaches, technical officials, NOCs, venues, events, schedules, medals, and medalists, forced our team to confront the full complexity of the database engineering lifecycle, from conceptual design all the way through physical implementation and query optimization. The following paragraphs summarize the most significant technical and collaborative lessons extracted from this experience.

- **Conceptual design is the most critical phase.** Early mistakes in the Entity-Relationship model propagated into every later stage. Designing a clean ER diagram — particularly the correct identification of weak entities, multi-valued attributes, and participation constraints among entities like ATHLETE, EVENT, and MEDAL, which required multiple iterations before a normalized schema could be derived. This reinforced the principle that time invested at the design stage saves significantly more time during implementation.

- **Normalization up to 3NF has real consequences.** Decomposing tables to eliminate transitive dependencies was not merely an academic exercise. For example, separating NOC information from athlete records, and venue details from schedule entries, prevented redundant updates and ensured referential integrity throughout the import process. The relational model only became stable once every table was audited against the third normal form criteria.

- **Data import is never trivial.** Loading nine CSV files into Oracle through temporary staging tables exposed a variety of real-world data quality issues: inconsistent date formats, null values in non-nullable columns, duplicate primary key candidates, and encoding anomalies in athlete names. Writing robust INSERT-SELECT scripts that handled these edge cases, without modifying the source CSV files, demanded careful use of Oracle's NVL, TO_DATE,

and TRIM functions, as well as a disciplined approach to constraint ordering during table creation.

- **SQL mastery goes beyond SELECT.** Answering the 30 validation questions required the deliberate use of the full range of SQL constructs covered in class: GROUP BY with HAVING, nested subqueries, correlated subqueries, analytic window functions, multi-table JOINs, set operations, and EXISTS clauses. Writing efficient queries, and recognizing when a simpler JOIN outperforms a subquery, was a skill that only developed through repeated practice and peer review within the team.

- **Triggers add a layer of database intelligence.** Implementing triggers to automate tasks such as medal count updates or schedule conflict detection highlighted how business logic can be enforced at the database level rather than relying solely on application code. Understanding the execution context of BEFORE and AFTER triggers, and avoiding mutating-table errors, were concrete Oracle-specific challenges that deepened the team's understanding of RDBMS internals.

This project demonstrated that database engineering is an iterative, holistic discipline. No stage exists in isolation: decisions made during conceptual design cascade directly into the relational model, which in turn shapes the physical schema, the import logic, and ultimately the expressiveness of the queries that can be written. The Olympic Games dataset, with its inherent complexity of many-to-many relationships between athletes, events, and nations, provided an ideal vehicle for exploring these dependencies in a meaningful real-world context.

Beyond the technical dimension, the project reinforced the importance of clear communication within a team and rigorous attention to academic integrity, particularly relevant given the explicit guidelines on the use of AI tools and the strict anti-plagiarism policy of the course. Documenting every decision, every design choice, and every tool used is not a bureaucratic formality but a fundamental practice of professional database development.

In summary, this project provided the team with a concrete, end-to-end experience of building a relational database from scratch: a challenge that connected theory to practice, individual effort to teamwork, and technical skill to professional responsibility. The competences acquired, ER modeling, normalization, Oracle SQL, data import engineering, and query design, constitute a solid foundation for our future in this degree.